সি ভাষার কম্পাইলার

সি ল্যাঙ্গুয়েজে প্রোগ্রাম লেখার জন্য প্রথমে কোন এডিটরে প্রোগ্রাম লিখে কম্পাইল করে এক্সিকিউটেবল ও ডিবাগিং করতে হয়। বর্তমানের কম্পাইলারগুলোতে এ কাজগুলো একসাথে করা যায়। তাই প্রোগ্রাম লেখার জন্য কম্পিউটারে সুবিধাজনক কোন কম্পাইলার ইন্সটল করে নিতে হবে। বর্তমানে অনেক ওপেন সোর্স কম্পাইলার ইন্টারনেট থেকে ফ্রি ডাউনলোড করা যায়। যেমন- কোডব্লক। এছাড়াও নেটবিনস, একলিপ্স, মাইক্রোসফট ভিজুয়াল স্টুডিওসহ বিভিন্ন সফটওয়্যার দিয়েও সি ভাষায় প্রোগ্রামিং করা যায়। এছাড়াও অ্যান্ড্রয়েড অপারেটিং সিস্টেম চালিত মোবাইল ফোনের জন্যও বিভিন্ন কম্পাইলার অ্যাপ (যেমন- C4droid,CppDroid) পাওয়া যায়।

প্রোগ্রাম কম্পাইলিং (Program Compiling)

হাই-লেভেল ল্যাঙ্গুয়েজে লিখিত প্রোগ্রামকে কম্পিউটারের বোধগম্য মেশিন ল্যাঙ্গুয়েজে রূপান্তরকে কম্পাইলিং বলা হয়। সি এর কম্পাইলারের এডিটরে সোর্স প্রোগ্রাম লেখার পর তা নির্বাহ করার জন্য প্রথমে তা কম্পাইল করতে হয়। সি প্রোগ্রাম কম্পাইল করার জন্য বিভিন্ন কম্পাইলারে কম্পাইল নির্দেশ দিতে হয়। সোর্স ফাইল কম্পাইল করলে তার সাথে সংযুক্ত অন্যান্য সোর্স ও হেডার ফাইলের সমন্বয়ে একই নামে একটি অবজেক্ট (.OBJ) ফাইল তৈরি হয়।

কম্পাইলিং-এর প্রধান কাজ (Main Functions of Compiling)

১. সোর্স প্রোগ্রামকে অনুবাদ করে অবজেক্ট প্রোগ্রাম তৈরি করা।
২. প্রোগ্রামে প্রয়োজনীয় রুটিন (প্রোগ্রামের ছোট অংশ)-কে লিংক করা।
৩. প্রোগ্রামের মধ্যে কোনো ভুল থাকলে তা জানানো।
৪. প্রোগ্রামের জন্য প্রধান মেমোরিতে অবস্থান তৈরি করা।
৫. প্রোগ্রামের মধ্যেকার ভুল-ত্রুটি সংশোধন করা।

সি কম্পাইলারে প্রোগ্রাম লেখার জন্য নির্দেশ দিয়ে নতুন উইন্ডোতে প্রোগ্রাম লিখে সেভ করতে হয়। এই ফাইলের এক্সটেনশন হবে C (যেমন-test_program.c)। সি প্রোগ্রাম নির্বাহ করার জন্য প্রথমে কম্পাইল, অতঃপর রান বা নির্বাহ করতে হয়। সোর্স ফাইল কম্পাইল করলে একই নামে একটি অবজেক্ট (.OBJ) ফাইল তৈরি হয়। কম্পাইল বা নির্বাহ প্রক্রিয়ায় প্রোগ্রামে কোনো ভুল বা অসংগতি থাকলে সেগুলো Warning ও Error মেসেজ আকারে প্রদর্শিত হয়। অত:পর এন্টার কী চেপে সোর্স প্রোগ্রামে ফেরত এসে ত্রুটি সংশোধন (এডিট) করতে হয়। এডিট করে এরর ফ্রি করার পর কম্পাইল ও রান করার নির্দেশ দিলে প্রোগ্রামটি নির্বাহ হবে।

সি প্রোগ্রাম কম্পাইল করার ধাপসমূহ

১. প্রথমে কম্পাইলারে ঢুকে এডিটিং স্ক্রিনে আসতে হবে। (কোডব্লক কম্পাইলারটি চালু করে File >New>Empty file নির্দেশ দিতে হবে (অথবা Ctrl-Shift-N চাপতে হবে।)
২.খালি স্ক্রিনে প্রোগ্রাম লিখে প্রোগ্রামটি সংরক্ষণ করতে হবে। এই ফাইলের এক্সটেনশন হবে C (যেমন-test_program.c)। একে Source Code বলে। (কোডব্লকে File মেনুতে ক্লিক করে Save নির্বাচন করে প্রদর্শিত Save As ডায়ালগ বক্সে ফাইলের নাম লিখে OK বাটনে ক্লিক করতে হবে।)
৩. এখন প্রোগ্রামটি কম্পাইল করার নির্দেশ দিতে হবে। (কোডব্লকে Compile মেনুতে ক্লিক করে এর অধীনে Compile অপশনটি সিলেক্ট করতে হবে অথবা ALt+F9 কী-দ্বয় চাপতে হবে। অথবা, Build মেনুতে ক্লিক করে Build and run নির্বাচন করতে হবে।)
৪. প্রোগ্রামটি কম্পাইল করার পর যদি এতে কোনো Error থাকে, তাহলে তা ঠিক করতে হবে।
৫. রান করার নির্দেশ দিতে হবে। অবজেক্ট (.OBJ) ফাইল থেকে এক্সিকিউটেবল (.EXE) ফাইল তৈরি হবে। একে প্রোগ্রাম নির্বাহ বলা হয়। (কোডব্লকে Build মেনুতে ক্লিক করে Build and run নির্বাচন করলে অথবা Build মেনুতে ক্লিক করে Run কমান্ড অপশনে ক্লিক করলে অথবা, Ctrl+F10 কী-দ্বয় চাপলে প্রোগ্রামটি রান হবে।)

IDE (Integrated Development Environment) হলো একটি সমন্বিত পরিবেশ, যেখানে প্রোগ্রামাররা সফটওয়্যার লেখা, এডিটিং করা, এক্সিকিউটেবল তৈরি করা ও ডিবাগিং করার সুযোগ পায়। সাধারণভাবে পূর্বে সি ল্যাঙ্গুয়েজে প্রথমে কোন এডিটরে প্রোগ্রাম লিখে পরবর্তীতে কম্পাইলারে নিয়ে এক্সিকিউটেবল ও ডিবাগিং করা হতো। কিন্তু বর্তমানে বেশিরভাগ কম্পাইলারে IDE সুবিধা থাকায় (যেমন- কোডব্লক, ডেভ সি++) কম্পাইলারেই কোড লেখা, সম্পাদনা করা, কম্পাইল করা এবং ডিবাগিং ও রান করা যায়।

ডেটা টাইপ (Data Type)

সি প্রোগ্রামে বিভিন্ন ধরনের ডেটা নিয়ে কাজ করা হয়। প্রোগ্রামে সাধারণত ডেটা ইনপুট করা হয় এবং প্রোগ্রাম ডেটা প্রসেস করে আউটপুট প্রদান করে থাকে। কোন ধরনের ডেটা ইনপুট নিবে প্রোগ্রামে তা আগেই উল্লেখ করে দিতে হয়। প্রথমে আমরা আমাদের তৈরি করা প্রোগ্রাম দুটিতে ব্যবহৃত ডেটার বিষয়টি দেখব এবং পরবর্তীতে ডেটা সম্পর্কে বিস্তারিত জানব। আমাদের তৈরিকৃত প্রোগ্রাম দুটি হলো :

```
/* Program of summation and average */
#include <stdio.h>
int main(){
    char name1[20];
    char name2[20];
    printf("Enter First name: ");
    scanf("%s",name1);
    printf("Enter Last name: ");
```

char ডেটা টাইপ এবং

```
mationa and average */
main(){
int a,b,c,sum;
float avg;
printf("Enter three integer Value :");
scanf("%d %d %d",&a,&b,&c);
sum = a+b+c;
avg=sum/3;
printf("\n The summation of three number is=%d",sum);
```

int এবং float ডেটা টাইপ

প্রোগ্রাম-২ : ছোট ও বড় হাতের অক্ষর প্রদর্শন

```
#include<stdio.h>
int main(){
char ch;
printf("Enter a character: ");
scanf("%c", &ch);
if((ch >= 'A') && (ch <= 'Z'))
printf("\nYou entered a capital letter : %c", ch);
else if((ch >= 'a') && (ch <= 'z'))
printf("\nYou entered a small letter : %c", ch);
else
printf("\nThe letter you entered is not a character.");
return 0;
}
```

সি প্রোগ্রামে ক্যারেক্টার টাইপ ডেটা (অর্থাৎ বিভিন্ন বর্ণ বা বর্ণমালা) নিয়ে কাজ করার জন্য character ডেটা টাইপ char ব্যবহৃত হয়। এর জন্য ১ বাইট জায়গা থাকে। সাধারণত কোনো অক্ষর বা চিহ্ন রাখার জন্য এ টাইপের ডেটা ব্যবহার করা হয়। তবে সেই অক্ষরটা ইংরেজি বর্ণমালার অক্ষর হতে হবে। অন্য ভাষার অক্ষর char টাইপের ভেরিয়েবলে রাখা যাবে না।

```
Enter a character: y  (এন্টার)
You entered a small letter : y
```

প্রোগ্রাম-৩ : এন্ট্রিকৃত অক্ষর প্রদর্শন করা

```
#include<stdio.h>
int main(){
char ch;
printf("Enter a letter: ");
ch = getchar();
printf("\nYou entered the letter : %c ", ch);
return 0;
}
```

উপরিউক্ত প্রোগ্রামে char ডেটা টাইপের জন্য printf এবং scanf ফাংশনের ভিতরে %c ফরমেট স্পেসিফায়ার ব্যবহার করা হয়েছে। getchar() ফাংশনটি দিয়ে char ডেটা টাইপের ডেটা ইনপুট করা যায়।

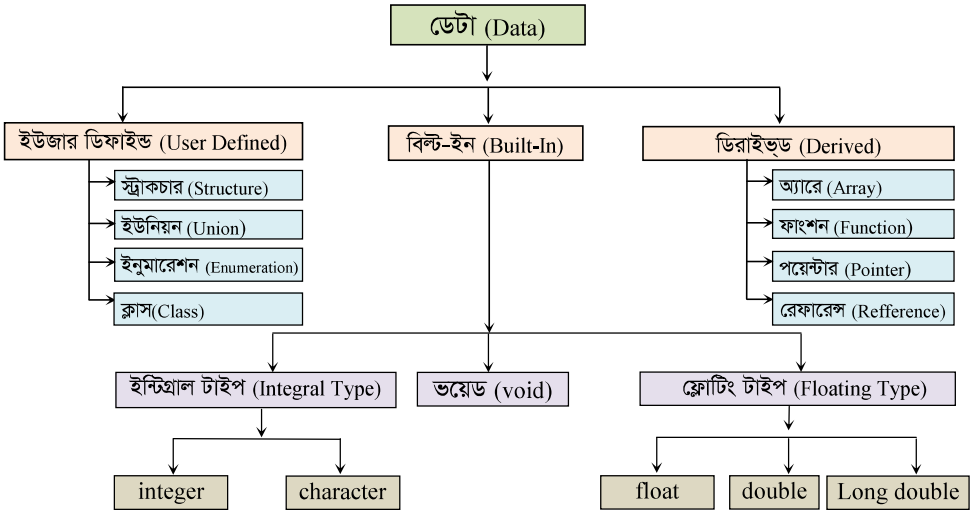
```
Enter a letter: s  (এন্টার)
You entered the letter : s
```

সি প্রোগ্রামিং-এ ভেরিয়েবল ঘোষণার জন্য ডেটার মান অনুযায়ী ডেটা টাইপ বলে দিতে হয়। বিভিন্ন টাইপের ডেটা মেমোরিতে ভিন্ন ভিন্ন বাইটের জায়গা দখল করে। ডেটার ধরন ও মেমোরি পরিসর সংরক্ষণের ভিত্তিতে সি ভাষায় চারটি মৌলিক ডেটা টাইপ আছে। যথা-

১. ক্যারেক্টার (Character)
২. ইন্টেজার (Integer)
৩. ফ্লোটিং পয়েন্ট (Floating Point)
৪. ডাবল (Double)

ফ্লোটিং টাইপের ডেটাকে Real ডেটা বলা হয়। যেমন- float, double, long double

এদেরকে মৌলিক বা বিল্ট-ইন ডেটা টাইপও বলা হয়। সি ল্যাঙ্গুয়েজে প্রয়োজনে নিজস্ব ডেটা টাইপ তৈরি করা যায়।



ছক : সি ল্যাঙ্গুয়েজ ডেটা টাইপ

সি-এর মৌলিক ডেটা টাইপগুলোর সংক্ষিপ্ত বর্ণনা

ডেটার ধরনকে ডেটা টাইপ বলা হয়। নিচে সি-এর মৌলিক চারটি ডেটা টাইপ-এর সংক্ষিপ্ত বর্ণনা দেয়া হলো।

১. **Character টাইপ :** সি প্রোগ্রামে ক্যারেক্টার টাইপ ডেটা (অর্থাৎ বিভিন্ন বর্ণ বা বর্ণমালা) নিয়ে কাজ করার জন্য character টাইপ ডেটা ব্যবহৃত হয়। character টাইপ ভেরিয়েবল ঘোষণার জন্য char কী-ওয়ার্ড লিখতে হয়। যেমন-

```
char x;
```

character টাইপ-এর জন্য 1 বাইট বা 8 বিট জায়গা থাকে। char টাইপ ডেটা ভ্যালু 0-255 এর মধ্যে সীমাবদ্ধ এবং 256 অক্ষর লেখা যায়। ক্যারেক্টার আবার Signed char ও Unsigned char ধরনের হয়। এর রেঞ্জ -128 থেকে +127 পর্যন্ত।

২. **Integer টাইপ :** সি প্রোগ্রামে পূর্ণসংখ্যা (যেমন : 23, 1435, 456, –235 ইত্যাদি) নিয়ে কাজ করার জন্য integer ব্যবহৃত হয়। ইন্টিজার টাইপের ভেরিয়েবল ঘোষণার জন্য int কীওয়ার্ড ব্যবহৃত হয়। যেমন-

```
int x;
```

মেমোরিতে integer লেখার জন্য 2 বাইট বা 16 বিট জায়গা নেয়। ইন্টিজার আবার Signed int ,Unsigned int , Short int ও Long int ধরনের হয়। এর রেঞ্জ -32768 থেকে +32767 পর্যন্ত।

৩. **Float টাইপ :** সি প্রোগ্রামে ভগ্নাংশ বা দশমিক চিহ্নযুক্ত সংখ্যা (যেমন : 23.45, –456.50, 2345.23 ইত্যাদি) নিয়ে কাজ করার জন্য float ডেটা টাইপ ব্যবহৃত হয়। float টাইপ ঘোষণার জন্য float কী-ওয়ার্ড ব্যবহৃত হয়। দশমিক চিহ্ন 6 ডিজিট পর্যন্ত গ্রহণযোগ্য।

```
float y;
```

প্রতিটি float টাইপ ভেরিয়েবলের জন্য কম্পাইলার মেমোরিতে 4 বাইট বা 32 বিট জায়গা সংরক্ষণ করে। এর রেঞ্জ -3.4xE-38 থেকে 3.4xE+38 পর্যন্ত।

৪. **Double টাইপ :** সি প্রোগ্রামে ফ্লোটের চেয়ে বড় দশমিক চিহ্নযুক্ত সংখ্যা বা এক্সপোনেন্সিয়াল বা সায়েন্টিফিক ফরম্যাটে সংখ্যা (যেমন : $3.5*10^{20}$, $-3.5*10^{200}$, $4.5*10^{-200}$ ইত্যাদি) নিয়ে কাজ করার জন্য double ডেটা টাইপ ব্যবহৃত

হয়। double টাইপ ঘোষণার জন্য double কী-ওয়ার্ড ব্যবহৃত হয়। double ডেটা টাইপে দশমিক চিহ্নের পর 14 ডিজিট পর্যন্ত গ্রহণযোগ্য। প্রতিটি double টাইপ মেমোরিতে 8 বাইট জায়গা লাগে।

```
double z;
```

float ডেটা টাইপ ছাড়া অন্যান্য বেসিক ডেটা টাইপের সাথে আবার signed, unsigned, short, long ইত্যাদি যুক্ত করে ডেটার রেঞ্জ এবং সংরক্ষণের জন্য মেমোরি পরিমাণ বাড়ানো-কমানো যায় বলে এদেরকে ডেটা টাইপ মডিফায়ার বলা হয়। মডিফায়ার যোগ করে ডেটার ধরন ও মেমোরি পরিসরের পরিমাণ বাড়ানো বা কমানো যায়। এ ধরনের কয়েকটি উদাহরণ হলো unsigned char, long, int ইত্যাদি। এদেরকে মডিফাইড ডেটা টাইপ বলা হয়।

সাধারণত signed ও unsigned মডিফায়ার char ও int টাইপ ভেরিয়েবলের জন্য এবং long ও short মডিফায়ার int ও double টাইপ ভেরিয়েবলের জন্য ব্যবহৃত হয়। এগুলো ছাড়াও সি ল্যাঙ্গুয়েজে আবার প্রয়োজনে নিজস্ব ডেটা টাইপ তৈরি করে নেয়া যায়। এরূপ ডেটা টাইপকে ইউজার ডিফাইন্ড বা কাস্টম ডেটা টাইপ বলা হয়। ভিন্ন ভিন্ন ডেটা টাইপের প্রয়োজনে একই ভেরিয়েবলের জন্য সংরক্ষিত মেমোরি পরিসর আলাদা হয়।

ডেটা টাইপ	সাইজ	ভেরিয়েবলের রেঞ্জ
char	1 byte	-128 to 127 বা, -2^7 to 2^{7-1}
signed char	1 byte	-128 to 127 বা, -2^7 to 2^{7-1}
unsigned char	1 byte	0 to 255 বা, 0 to (2^{8-1})
int	2/4 byte	-32768 to 32767 বা, -2^{15} to 2^{15-1}
signed int	2 byte	-32768 to 32767 বা, -2^{15} to 2^{15-1}
unsigned int	2 byte	0 to 65535 বা, 0 to (2^{16-1})
short int	2 byte	-32768 to 32767
long int	4 byte	-2147483648 to 2147483647 বা, -2^{31} to 2^{31-1}
signed long	4 byte	-2147483648 to 2147483647
unsigned long	4 byte	0 to 4294967295 বা, 0 to (2^{32-1})
float	4 byte	3.4xE-38 to 3.4xE+38
double	8 byte	1.7xE-308 to 1.7xE+308
long double	10 byte	3.4xE-4932 to 1.1xE+4932

ডেটা টাইপ ঘোষণার নিয়ম

- ভেরিয়েবল ঘোষণার আগে ডেটা টাইপ নির্ধারণ করার জন্য ডেটা টাইপের কী-ওয়ার্ড লিখতে হবে। যেমন- int, float,
- তারপর স্পেস দিয়ে ভেরিয়েবল-এর নাম লিখতে হবে। যেমন- int sum
- সবশেষে সেমিকোলন দিতে হবে। যেমন- int sum;
- একই টাইপের একাধিক ভেরিয়েবল ঘোষণায় ডেটা টাইপ কী-ওয়ার্ডের পর একই লাইনে কমা দিয়ে ভেরিয়েবলসমূহকে আলাদা করে লিখে সবশেষে সেমিকোলন লিখতে হবে। যেমন- int n1, n2, sum ইত্যাদি।
- ঘোষণাকৃত ভেরিয়েবল টাইপ অনুযায়ী স্টেটমেন্টে ফরমেট স্পেসিফায়ার ব্যবহার করতে হয়।

ফর্মেট :

```
< ডেটা টাইপ কীওয়ার্ড > < ভেরিয়েবল নাম >;     অথবা,
< ডেটা টাইপ কীওয়ার্ড > < ভেরিয়েবল নাম১ >, < ভেরিয়েবল নাম২ >, < ভেরিয়েবল নাম১ >;
```

ডেটা টাইপ	ব্যবহৃত কী-ওয়ার্ড	উদাহরণ	
Character	char	char a;	char a,b,c;
Integer	int	int number;	int number1, number2;
Float	float	float a;	float a,b,c;
Double	double	double x;	double x,y,z;

ধ্রুবক (Constant)

ধ্রুবক (Constant) : প্রোগ্রাম নির্বাহের সময় সি প্রোগ্রামিং ভাষায় এমন কিছু মান আছে, যা কখনো পরিবর্তন হয় না। যেমন- পাই (π) এর মান হলো 3.1416 যা কখনো পরিবর্তন হয় না। প্রোগ্রাম নির্বাহের সময় যে রাশির মান অপরিবর্তিত থাকে, তাকে কন্সট্যান্ট বা ধ্রুবক বলা হয়। সি প্রোগ্রামিং ভাষায় দুইভাবে কন্সট্যান্ট ঘোষণা করা যায়। যথা-

১. const কী-ওয়ার্ড ব্যবহার করে
২. #define প্রিপ্রসেসর ব্যবহার করে

const কী-ওয়ার্ড ব্যবহার করে ধ্রুবক ঘোষণার ফরম্যাট :

```
const  ConstType  ConstName = ConstValue;
```

যেমন:
```
const float  pi   =3.1416;
const  int num  = 40;
const  float val1= 40.7523;
```

#define প্রিপ্রসেসর ব্যবহার করে ধ্রুবক ঘোষণার ফরম্যাট :

```
#define ConstName ConstValue
```

যেমন:
```
#define pi 3.1416
#define num 40
#define name " Ebadur Rahman"
```

কন্সট্যান্ট ব্যবহারের নিয়ম

কন্সট্যান্ট ব্যবহারের কতগুলো সুনির্দিষ্ট নিয়ম আছে। যেমন-

১. প্রতিটি কন্সট্যান্টের নাম থাকে।
২. কন্সট্যান্ট ঘোষণার সময়ই তার মান নির্ধারণ করে দিতে হয়।
৩. প্রোগ্রাম নির্বাহের সময় কোনো অবস্থাতেই মান পরিবর্তন করা যায় না।
৪. প্রয়োজনে প্রোগ্রামের যেকোনো জায়গায় কন্সট্যান্ট ব্যবহার করা যায়।
৫. কন্সট্যান্টের মান প্রদর্শনের জন্য উপযুক্ত ফরম্যাট স্পেসিফায়ার ব্যবহৃত হয়।

প্রোগ্রাম-৪ : বিভিন্ন কন্সট্যান্ট-এর ব্যবহার

```
#include<stdio.h>
int main(){
const int height = 100;          /*int constant*/
const float number = 3.14;   /*Real constant*/
const char letter = 'A';         /*char constant*/
const char letter_sequence[10] = "ABC"; /*string constant*/
printf("value of height :%d \n", height);
printf("value of number : %f \n", number );
printf("value of letter : %c \n", letter );
printf("value of letter_sequence : %s \n", letter_sequence);
return 0;
}
```

প্রোগ্রামটিতে তিন ধরনের ভেরিয়েবল টাইপ ব্যবহার করা হয়েছে। সংখ্যা রাখার জন্য int, ভগ্নাংশ যুক্ত সংখ্যা রাখার জন্য float এবং ক্যারেক্টার রাখার জন্য char টাইপ ব্যবহার করা হয়েছে। char টাইপের সাথে [10] ব্যবহার করায় দশটি অক্ষর রাখা যাবে।

আউটপুট :

```
value of height : 100
value of number : 3.140000
value of letter : A
value of letter_sequence : ABC
```

Backslash Character : C প্রোগ্রামে কিছু Backslash Character ব্যবহার করা হয়, যা আউটপুট ফাংশন হিসেবে কাজ করে। সাধারণত ব্যাকস্ল্যাশ (\) দেয়ার পর একটি ক্যারেক্টার ব্যবহার করা হয়, যা একটি নির্দিষ্ট ফাংশন হিসেবে কাজ করে। এগুলোকে ইস্কেপ সিকুয়েন্স (escape sequence) বলা হয়। প্রতিটি ব্যাকস্ল্যাশ ক্যারেক্টার কন্সট্যান্ট আলাদা আলাদা কাজে ব্যবহার করা হয়। যেমন- avg ভেরিয়েবলের মান নতুন লাইনে প্রদর্শন করানোর জন্য আউটপুট ফাংশন printf এর সাথে প্রোগ্রামে নিচের মতো স্টেটমেন্ট লিখতে হবে।

```
printf ("avg is %f\n", avg);
```

এখানে, %f-হলো ফরমেট স্পেসিফায়ার, যা ফ্লোটিং টাইপের ডেটা আউটপুট করার জন্য ব্যবহৃত হয়।
\n-হলো ইস্কেপ সিকুয়েন্স, যা নতুন লাইনের শুরুতে আউটপুট প্রদর্শনের জন্য ব্যবহৃত হয়।

নিচের টেবিলে ব্যাকস্ল্যাশ ক্যারেক্টার কন্সট্যান্ট ও তাদের কাজ দেখানো হলো।

ব্যাকস্ল্যাশ ক্যারেক্টার	বর্ণনা
\a	audible alert (bell) অর্থাৎ সতর্ক সংকেত প্রদান করার জন্য ব্যবহৃত হয়।
\b	back space অর্থাৎ কার্সরকে বামে এক ঘর সরানোর জন্য ব্যবহৃত হয়।
\f	form feed অর্থাৎ আউটপুট নিচের লাইনে প্রদর্শনের জন্য ব্যবহৃত হয়।
\n	new line অর্থাৎ নতুন লাইনের শুরুতে প্রদর্শনের জন্য ব্যবহৃত হয়।
\r	carriage return অর্থাৎ আউটপুট কোন লাইনের টেক্সটের শুরুতে প্রদর্শনের জন্য ব্যবহৃত হয়।
\t	horizontal tab অর্থাৎ আউটপুট ডানদিকে এক ট্যাব পরিমাণ দূরে প্রদর্শনের জন্য ব্যবহৃত হয়।

প্রোগ্রাম-৫ : ইস্কেপ সিকুয়েন্স (escape sequence)-এর ব্যবহার

```
//using escape sequence \n
#include<stdio.h>
int main(){
printf("SDL is a leading \n software company in BD");
return 0;
}
```

\n - ইস্কেপ সিকুয়েন্স

আউটপুট নতুন লাইনের শুরুতে প্রদর্শনের জন্য \n ইস্কেপ সিকুয়েন্স ব্যবহার করা হয়। বড় কোন লেখার মাঝখানে \n লিখলে-এর পরবর্তী লেখাসমূহ নিচের লাইনে অর্থাৎ নতুন লাইন হিসেবে প্রদর্শিত হবে। প্রোগ্রামে একটি ইস্কেপ সিকুয়েন্স \n ব্যবহারের কারণে আউটপুট হবে :

```
SDL is a leading
software company in BD.
```

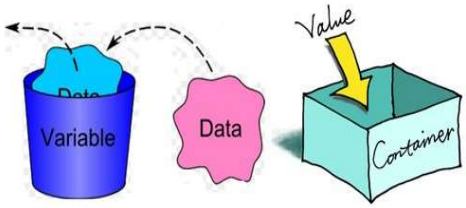
চলক (Variable)

সি ভাষায় মেমোরিতে ডেটা সংরক্ষণ করতে যে নাম ব্যবহৃত হয়, তাকে চলক (Variable) বলা হয়। চলক (Variable) হলো মেমোরি লোকেশনের নাম বা ঠিকানা। প্রোগ্রামে যখন কোনো ডেটা নিয়ে কাজ করা হয়, প্রাথমিকভাবে সেগুলো কম্পিউটারের মেমোরিতে (র‍্যামে) রাখা হয়। এক্ষেত্রে মেমোরি অ্যাড্রেস সরাসরি ব্যবহার না করে একটি নাম দিয়ে ঐ নামের অধীনে ডেটা রাখা হয়। ঐ নামকেই ভেরিয়েবল বলা হয়। প্রতিবার প্রোগ্রাম নির্বাহের সময় মেমোরিতে ভেরিয়েবলগুলোর অবস্থান এবং এতে সংরক্ষিত মান পরিবর্তন হয় বা হতে পারে বলে এদেরকে ভেরিয়েবল বা চলক বলা হয়। ব্যবহারকারীর প্রয়োজনানুযায়ী মান ব্যবহার এবং ফলাফল পেতে ভেরিয়েবল ব্যবহৃত হয়।

```
#include <stdio.h>
main(){
int a,b,c, Sum;
float Avg;
printf("Enter three integer value:");
scanf("%d %d %d",&a,&b,&c);
Sum = a+b+c;
Avg = Sum/3;
printf("\nSum  =%d", Sum);
printf("\nAverage=%.2f", Avg);
}
```

ডেটার ধরনের ওপর ভিত্তি করে সি ল্যাঙ্গুয়েজে তিন ধরনের ভেরিয়েবল ব্যবহার করা হয়। যথা-

১. ক্যারেক্টার ভেরিয়েবল (Character Variable)
২. ইন্টিজার ভেরিয়েবল (Integer Variable)
৩. ফ্লোটিং পয়েন্ট ভেরিয়েবল (Floating Point Variable)



ভেরিয়েবল হলো একটা ধারক বা পাত্রের মতো। যেমন-এক লিটার পানি সংরক্ষণ করার জন্য একটি পাত্র দরকার হবে, যার ধারণক্ষমতা ১ লিটার বা ১ লিটার থেকে বেশি। একইভাবে প্রোগ্রামিং করার সময় যখন কোনো ডেটা সংরক্ষণ করার দরকার হয়, তখন আমাদের মেমোরির একটি জায়গা বা স্পেস দরকার হয়। এই মেমোরি স্পেসকেই প্রোগ্রামের ভাষায় ভেরিয়েবল বলা হয়। একই লাইনে এভাবে ভেরিয়েবল ঘোষণা করা হলে মাঝখানে কমা এবং শেষে সেমিকোলন ব্যবহার করতে হয়।

ক্যারেক্টার ভেরিয়েবল (Character Variable)

নাম, ঠিকানা বা যে কোনো বর্ণের ক্ষেত্রে ক্যারেক্টার ভেরিয়েবল ব্যবহৃত হয়। এর জন্য 1 বাইট মেমোরি লাগে। ক্যারেক্টার টাইপের ভেরিয়েবল ঘোষণার আগে char ব্যবহৃত হয়। যেমন- char a, char x, char name, char address ইত্যাদি।

ইন্টিজার ভেরিয়েবল (Integer Variable)

সি ভাষায় পূর্ণ সংখ্যা নিয়ে কাজ করার জন্য ইন্টিজার ভেরিয়েবল ব্যবহৃত হয়। এর জন্য 2 বাইট মেমোরি লাগে। ইন্টিজার টাইপের ভেরিয়েবল ঘোষণার জন্য int ব্যবহৃত হয়। যেমন- int a, int x, int salary ইত্যাদি।

ফ্লোটিং পয়েন্ট ভেরিয়েবল (Floating Point Variable)

সি ভাষায় দশমিক সংখ্যা বা প্রকৃত সংখ্যার ক্ষেত্রে ফ্লোটিং পয়েন্ট ভেরিয়েবল ব্যবহৃত হয়। এর জন্য 4 বাইট মেমোরি লাগে। ফ্লোটিং পয়েন্ট টাইপের ভেরিয়েবল ঘোষণার জন্য float ব্যবহৃত হয়। যেমন- float a, float x ইত্যাদি।

```
#include <stdio.h>
main(){
int a,b,c, Sum
float Avg;
printf("Enter three integer value:");
scanf("%d %d %d",&a,&b,&c);
Sum = a+b+c;
Avg = Sum/3;
printf("\nSum    =%d", Sum);
printf("\nAverage=%.2f", Avg);
}
```

প্রোগ্রামে ব্যবহৃত ভেরিয়েবল

ডিক্লারেশনের ওপর ভিত্তি করে ভেরিয়েবলের প্রকারভেদ

ডিক্লারেশনের ওপর ভিত্তি করে ভেরিয়েবলকে দুই ভাগে ভাগ করা যায়।
যথা- ১। লোকাল ভেরিয়েবল (Local Variable)
২। গ্লোবাল ভেরিয়েবল (Global Variable)

১. লোকাল ভেরিয়েবল (Local Variable)

কোন ফাংশনের মধ্যে ভেরিয়েবল ডিক্লেয়ার করলে তাকে উক্ত ফাংশনের লোকাল ভেরিয়েবল বলা হয়। কোন ফাংশনের মধ্যে ডিক্লেয়ার করা লোকাল ভেরিয়েবল উক্ত ফাংশনের বাইরে ব্যবহার করা যায় না। লোকাল ভেরিয়েবলের কর্মকাণ্ড শুধুমাত্র সংশ্লিষ্ট ফাংশনেই সীমাবদ্ধ থাকে। ভিন্ন ভিন্ন ফাংশনে একই নামের লোকাল ভেরিয়েবল থাকতে পারে।

২. গ্লোবাল ভেরিয়েবল (Global Variable)

সকল ফাংশনের বাইরে প্রোগ্রামের শুরুতে ডিক্লেয়ার করা ভেরিয়েবলকে গ্লোবাল ভেরিয়েবল বলা হয়। গ্লোবাল ভেরিয়েবল সাধারণত প্রোগ্রামের শুরুতেই ডিক্লেয়ার করা হয়। গ্লোবাল ভেরিয়েবলের কর্মকাণ্ড কোন ফাংশনের মধ্যে সীমাবদ্ধ নয়। লোকাল ভেরিয়েবল ও গ্লোবাল ভেরিয়েবলের ব্যবহার দেখার জন্য নিচের প্রোগ্রামটি লক্ষ করা যাক। বৃত্তের ক্ষেত্রফল নির্ণয়ের জন্য scanf() ব্যবহার করে কীবোর্ড হতে বৃত্তের ব্যাসার্ধ গ্রহণ করতে হবে। অতঃপর ক্ষেত্রফল = $\pi \times$ (ব্যাসার্ধ)^২ সূত্র ব্যবহার করে বৃত্তের ক্ষেত্রফল নির্ণয় করতে হবে। এখানে π (pi) একটি ধ্রুব সংখ্যা, যার মান 22/7=3.14।

প্রোগ্রাম-৬: বৃত্তের ক্ষেত্রফল নির্ণয় করা (ভেরিয়েবলের ব্যবহার)

```
#include <stdio.h>
float pi=3.14;
int main(){
float radius, area;
printf("Enter radius of the circle (in cm):");
scanf("%f", &radius);
area= (pi)*(radius*radius);
printf("Area of the circle is : %.2f square cm", area);
return 0;
```

গ্লোবাল ভেরিয়েবল

লোকাল ভেরিয়েবল

এখানে float pi হলো গ্লোবাল ভেরিয়েবল। এটি main() ফাংশনের আগে ডিক্লেয়ার করায় প্রোগ্রামের যে কোনো জায়গায় ব্যবহার করা যাবে। radius ও area ভেরিয়েবল main() ফাংশনের ভিতরে হওয়ায় এগুলো লোকাল ভেরিয়েবল। লোকাল ভেরিয়েবল উক্ত ফাংশনের বাইরে ব্যবহার করা যায় না। লোকাল ভেরিয়েবলের কর্মকাণ্ড শুধুমাত্র সংশ্লিষ্ট ফাংশনেই সীমাবদ্ধ থাকে।

লোকাল ভেরিয়েবল ও গ্লোবাল ভেরিয়েবল-এর মধ্যে পার্থক্য

লোকাল ভেরিয়েবল	গ্লোবাল ভেরিয়েবল
১. কোন ফাংশনের মধ্যে ভেরিয়েবল ডিক্লেয়ার করলে, তাকে উক্ত ফাংশনের লোকাল ভেরিয়েবল বলা হয়।	১. সকল ফাংশনের বাইরে প্রোগ্রামের শুরুতে ডিক্লেয়ার করা ভেরিয়েবলকে গ্লোবাল ভেরিয়েবল বলা হয়।
২. লোকাল ভেরিয়েবল যে ফাংশনের মধ্যে ডিক্লেয়ার করা হয়, তার বাইরে ব্যবহার করা যায় না।	২. গ্লোবাল ভেরিয়েবলের কার্যক্রম কোন ফাংশনের মধ্যে সীমাবদ্ধ নয়। অর্থাৎ পুরো প্রোগ্রামের জন্য প্রযোজ্য।
৩. ভিন্ন ভিন্ন ফাংশনে একই নামের লোকাল ভেরিয়েবল থাকতে পারে।	৩. একটি প্রোগ্রামে একই নামের একটিমাত্র গ্লোবাল ভেরিয়েবল থাকতে পারে।
৪. লোকাল ভেরিয়েবল ফাংশনের শুরুতে লিখতে হয়।	৪. গ্লোবাল ভেরিয়েবল প্রোগ্রামের শুরুতে লিখতে হয়।

ভেরিয়েবল নামকরণের নিয়ম

প্রোগ্রামের শুরুতে প্রয়োজনীয় সংখ্যক ভেরিয়েবল ঘোষণা করে প্রোগ্রামের পরবর্তী অংশে সেগুলো ব্যবহার করতে হয়। তবে ভেরিয়েবল তৈরি (ঘোষণা) এবং নামকরণের মধ্যে কিছু মৌলিক সীমাবদ্ধতা ও নিয়ম-কানুন রয়েছে। নিম্নে সেগুলোর কয়েকটি উল্লেখ করা হলো :

১. ভেরিয়েবলের প্রথম অক্ষরটি অবশ্যই বর্ণ (a,..,z, A,..,Z) হতে হবে।

২. ভেরিয়েবলের প্রথম অক্ষরটির পরে ভেরিয়েবল নামকরণে কেবল আলফাবেটিক ক্যারেক্টার (a,..,z, A,..,Z), ডিজিট (0,..,9) এবং আন্ডারস্কোর (_) ও ডলার চিহ্ন ($) ব্যতীত অন্য কোন ক্যারেক্টার ব্যবহার করা যায় না।

৩. একই ফাংশনে একই নামে দুই বা ততোধিক ভেরিয়েবল ঘোষণা করা যায় না।

৪. ভেরিয়েবল নামের মধ্যে কোন ফাঁকা স্থান থাকতে পারে না।

৫. ভেরিয়েবল নাম ডিজিট বা অঙ্ক দিয়ে শুরু হতে পারে না।

৬. ভেরিয়েবলের নামকরণে ইংরেজি ছোট ও বড় হাতের অক্ষরগুলো আলাদা অর্থ বহন করে। তাই MyRoll নামে ভেরিয়েবল ঘোষণা করে একে আবার Myroll নামে ব্যবহার করা যায় না।

৭. কী-ওয়ার্ড, ফাংশন, স্টেটমেন্ট, রিজার্ভড ওয়ার্ড ইত্যাদির নাম ভেরিয়েবল হিসেবে ব্যবহার করা যায় না।

৮. ভেরিয়েবল নামকরণে সর্বাধিক 31 ক্যারেক্টার ব্যবহার করা যায়।

কয়েকটি বৈধ ও অবৈধ ভেরিয়েবলের উদাহরণ

অবৈধ ভেরিয়েবল	কারণ	বৈধ ভেরিয়েবল
int 5X;	বর্ণ দিয়ে শুরু হয়নি।	int X5;
int "Roll";	কোটেশন হবে না।	int Roll;
int main;	main ব্যবহার করা যাবে না।	int Main;
int Roll-10;	- চিহ্নের ব্যবহার বৈধ নয়।	int Roll_10;
float Marks 10;	মাঝে ফাঁকা থাকবে না।	float Marks10;

সি ল্যাঙ্গুয়েজে কন্সট্যান্ট ও ভেরিয়েবল-এর মধ্যে পার্থক্য

কনস্ট্যান্ট (Constant)	ভেরিয়েবল (Variable)
১. কন্সট্যান্টে একটি নির্দিষ্ট মান ধারণ করে।	১. ভেরিয়েবল হলো ডেটা রাখার জন্য মেমোরিতে লোকেশনের অ্যাড্রেস।
২. কন্সট্যান্টে কমা ব্যবহার করা যায় না, তবে প্রয়োজনে দশমিক ব্যবহার করা যায়।	২. ভেরিয়েবলের মান নির্ধারণ করার সময় সংখ্যার মধ্যে কমা ব্যবহার করা যাবে।
৩. প্রোগ্রাম চালানোর সময় কন্সট্যান্ট–এর মান পরিবর্তন করা যায় না।	৩. প্রোগ্রাম চালানোর সময় ইচ্ছেমতো ভেরিয়েবল–এর মান পরিবর্তন করা যায়।
৪. প্রোগ্রামে ধ্রুবক কম ব্যবহৃত হয়।	৪. প্রোগ্রামে চলক সবচেয়ে বেশি ব্যবহৃত হয়।
৫. উদাহরণ : const int a=55;	৫. উদাহরণ : int a;

অপারেটর (Operator)

প্রোগ্রামিং ভাষায় গাণিতিক ও যৌক্তিক কাজ নিয়ন্ত্রণ করার জন্য কতগুলো বিশেষ চিহ্ন ব্যবহৃত হয়, এগুলোকে অপারেটর বলা হয়। যেমন, +, -, *, /, <, > ইত্যাদি। এই অপারেটরসমূহ না থাকলে সি ভাষায় কখনো গাণিতিক বা যৌক্তিক করা সম্ভব হতো না। নিচে বিভিন্ন ধরনের অপারেটর সম্পর্কে আলোচনা করা হলো :

অ্যারিথমেটিক অপারেটর (Airthmetic Operator)

সি প্রোগ্রামে বিভিন্ন গাণিতিক কাজ (যেমন: যোগ, বিয়োগ, গুণ, ভাগ ইত্যাদি) সম্পন্ন করার জন্য যেসব সিম্বল বা অপারেটর ব্যবহৃত হয়, তাদেরকে অ্যারিথমেটিক বা গাণিতিক অপারেটর বলা হয়। পাশের ছকে বহুল ব্যবহৃত অ্যারিথমেটিক অপারেটরসমূহের তালিকা ও ব্যবহার উল্লেখ করা হলো।

অপারেটর	ব্যবহার	উদাহরণ	ফলাফল
/ (ডিভিশন)	ভাগ করার জন্য	81/9	9
* (মাল্টিপ্লিকেশন)	গুণ করার জন্য	7*6	42
% (মডিউল)	ভাগশেষ নির্ণয়	20%6	2
+ (বাইনারি প্লাস)	যোগ করা	8+5+2.75	15.75
- (বাইনারি মাইনাস)	বিয়োগ করা	9-5	4

টেবিল : অ্যারিথমেটিক অপারেটর

রিলেশনাল অপারেটর (Relational Operator)

সি প্রোগ্রামে দু'টো অপারেন্ডের মধ্যে সম্পর্ক (যেমন : ছোট, ছোট বা সমান, বড়, বড় বা সমান ইত্যাদি) বোঝানোর জন্য যে অপারেটর ব্যবহৃত হয়, তাকে রিলেশনাল অপারেটর বলা হয়। বস্তুত এই অপারেটর সংশ্লিষ্ট দুই অপারেন্ডের মধ্যে তুলনা

অপারেটর	উদাহরণ	ব্যবহার
|| (লজিক্যাল অর)	(A || B) is true	দু'টি অপারেন্ডের মধ্যে লজিক্যাল OR অপারেশনের জন্য।
&& (লজিক্যাল অ্যান্ড)	(A && B) is false	দু'টি অপারেন্ডের মধ্যে লজিক্যাল AND অপারেশনের জন্য।

করে ফলাফল সত্য/মিথ্যা হিসেবে জানিয়ে দেয়। নিম্নের ছকে বহুল ব্যবহৃত রিলেশনাল অপারেটরসমূহের তালিকা ও ব্যবহার উল্লেখ করা হলো :

অপারেটর	অর্থ
<	Less than (ছোট)
<=	Less than or equal (ছোট বা সমান)
>	Greater than (বড়)
>=	Greater than or equal (বড় বা সমান)
==	Equal to (সমান)
!=	Not equal to (অসমান)

টেবিল : রিলেশনাল অপারেটর

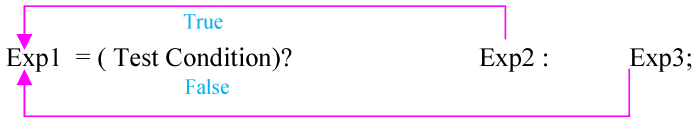
টেবিল : লজিক্যাল অপারেটর

লজিক্যাল অপারেটর (Logical Operator)

সি প্রোগ্রামে লজিক্যাল অপারেশন (যেমন-লজিক্যাল অর, লজিক্যাল অ্যান্ড, লজিক্যাল নট) সম্পন্ন করার জন্য যে অপারেটর ব্যবহৃত হয়, তাকে লজিক্যাল অপারেটর বলা হয়। লজিক্যাল অর ও লজিক্যাল অ্যান্ড অপারেটর হলো বাইনারি অপারেটর এবং এদের উভয় অপারেন্ড int টাইপ হয়। লজিক্যাল অর (||) ও লজিক্যাল অ্যান্ড (&&)-এর জন্য দু'টি করে অপারেন্ড থাকে, কিন্তু লজিক্যাল নট (!) এর জন্য একটি অপারেন্ড থাকে। এজন্য লজিক্যাল নট (!) একটি ইউনারি অপারেটর।

কন্ডিশনাল অপারেটর (Conditional Operator)

সি প্রোগ্রামে শর্ত সাপেক্ষে কোনো ভেরিয়েবল বা এক্সপ্রেশনের মান অন্য কোনো ভেরিয়েবল বা এক্সপ্রেশনের মান হিসেবে নির্ধারণ করার জন্য কন্ডিশনাল অপারেটর (?:) ব্যবহৃত হয়। কন্ডিশনাল অপারেটর ব্যবহারের ফরম্যাট হলো-



এখানে স্টেটমেন্টে প্রথমে Test Condition পরীক্ষিত হবে; এই মান সত্য বা অশূন্য হলে Exp1 = Exp2 নির্ধারিত বা সম্পাদিত হবে, আর এই মান মিথ্যা বা শূন্য হলে Exp1= Exp3 নির্ধারিত বা সম্পাদিত হবে।

ইনক্রিমেন্ট ও ডিক্রিমেন্ট অপারেটর (Increment and Decrement Operator)

ইনক্রিমেন্ট অপারেটর (Increment Operator) : প্রোগ্রামে কখনও কখনও ভেরিয়েবলের মান একটা নির্দিষ্ট ইনক্রিমেন্ট বাড়াতে হয়। ভেরিয়েবলের মানকে বর্ধিত (Increment) করার জন্যই মূলত ইনক্রিমেন্ট অপারেটর ব্যবহৃত হয়ে থাকে। এই বর্ধিতকরণ 1 থেকে শুরু করে যে কোনো মান হতে পারে। যেমন : x = x + 1; x = x + 2, x = x + 3 ইত্যাদি। ইনক্রিমেন্টের মান 1 করে বর্ধিত করার জন্য সি এর স্পেশাল ইনক্রিমেন্ট অপারেটর যেমন ++ ব্যবহার করা হয়। এ ধরনের অপারেটর একটিমাত্র অপারেন্ড নিয়ে কাজ করে বলে এদেরকে ইউনারি অপারেটর বলা হয়। যার সাধারণ গঠন (General Syntex) হলো : variable ++ বা ++ variable;

প্রথমটিকে Postfix ইনক্রিমেন্ট আর দ্বিতীয়টিকে Prefix ইনক্রিমেন্ট বলা হয়। সাধারণত while, for ইত্যাদি লুপিং-এর ক্ষেত্রে এ ধরনের ইনক্রিমেন্ট ব্যবহৃত হয়ে থাকে। অবশ্য Postfix ও Prefix-এর সাথে আলাদা আলাদা অর্থ প্রকাশ করে। যেমন, i = 4 এবং j = i++; এ ক্ষেত্রে i এর মান হবে 5, কিন্তু j এর মান হবে 4; আবার যদি i = 4 এবং j = ++ i; লেখা হয় তাহলে i ও j উভয়ের মান হবে 5 অর্থাৎ, Prefix প্রথমে বাম প্রান্তের variable-এর সাথে 1 যোগ করে, তারপর ফলাফলকে বাম প্রান্তের ভেরিয়েবল দ্বারা অ্যাসাইন করে। অপরদিকে Postfix প্রথমে বাম প্রান্তের variable-এর মান অ্যাসাইন করে, তারপর অপারেন্ডের মান 1 বর্ধিত করে।

অর্থাৎ i++ এর মানে হলো i এর মান 1 বাড়বে, কিন্তু আগের মানটিই ফেরত দিবে। যেমন-

```
#include<stdio.h>
```

এখানে, j = i++; লেখার কারণে i এর মান 1 বেড়ে 2 হবে, কিন্তু যেহেতু i++ আগের মানটি (1) ফেরত দেবে, তাই j এর মান হবে 1। প্রোগ্রামটি লিখে রান করালে আমরা ফলাফল দেখতে পাবো-

Output :
i =2
j =1

প্রোগ্রাম-৭: ইনক্রিমেন্ট অপারেটরের ব্যবহার

```
int main()
{
   int j;
   int i=1;
   j = i ++;
   printf("i=%d\n j=%d",i,j);
return 0;
}
```

++i এর মানে হলো i এর মান 1 বাড়বে এবং বাড়ার পর নতুন যে মান হবে, সে মানটিই ফেরত দিবে। যেমন-

প্রোগ্রাম-৮: ইনক্রিমেন্ট অপারেটরের ব্যবহার

```
#include<stdio.h>
int main()
{
   int j;
   int i=1;
   j = ++i;
   printf("i=%d\n j=%d",i,j);
return 0;
}
```

এখানে, j = ++i লেখার কারণে i এর মান 1 বেড়ে 2 হবে এবং বাড়ার পর নতুন যে মান হবে, সে মানটিই ফেরত দিবে। তাই j এর মান হবে 2। প্রোগ্রামটি লিখে রান করালে আমরা ফলাফল দেখতে পাবো-

Output :

```
i =2
j =2
```

ডিক্রিমেন্ট অপারেটর (Decrement Operator) : ইনক্রিমেন্ট অপারেটরের বিপরীতে কাজ করে ডিক্রিমেন্ট অপারেটর। অর্থাৎ ভেরিয়েবলের মানকে হ্রাস (Decrement) করার জন্যই মূলত ডিক্রিমেন্ট অপারেটর ব্যবহৃত হয়ে থাকে। এই হ্রাসকরণ 1 থেকে শুরু করে যে কোন মান হতে পারে। যেমন : x = x-1;x = x-2; x = x-3 ইত্যাদি। ডিক্রিমেন্টের মান 1 করে হ্রাস করার জন্য সি এর স্পেশাল ডিক্রিমেন্ট অপারেটর ব্যবহার করা হয় যার সাধারণ গঠন (Syntax) হলো :

variable-- অথবা --variable;

প্রথমটিকে postfix ডিক্রিমেন্ট এবং দ্বিতীয়টিকে prefix ডিক্রিমেন্ট বলা হয়। সাধারণত for, while লুপিং-এর ক্ষেত্রে এ ধরনের ডিক্রিমেন্ট ব্যবহৃত হয়ে থাকে।

variable ++ ও ++ variable এর মধ্যে পার্থক্য

variable ++ অথবা i++	++ variable অথবা ++i
১. একে Postfix বলা হয়।	১. একে Prefix বলা হয়।
২. Postfix প্রথমে বাম প্রান্ত variable-এর মান অ্যাসাইন করে, তারপর অপারেন্ডের মান 1 বর্ধিত করে।	২. Prefix প্রথমে Operand-এর সাথে 1 যোগ করে, তারপর ফলাফলকে বাম প্রান্তে ভেরিয়েবল দ্বারা অ্যাসাইন কর।
৩. উদাহরণ: i=4 এবং j= i++; এ ক্ষেত্রে i এর মান হবে 5 কিন্তু j এর মান হবে 4।	৩. উদাহরণ: i=4 এবং j=++i; এ ক্ষেত্রে i এবং j উভয়ের মান হবে 5।

স্পেশাল অপারেটর (Special Operator) : সি প্রোগ্রামে বহুল ব্যবহৃত আরও কিছু অপারেটর হলো কমা অপারেটর (,), সাইজ অফ অপারেটর (sizeof), পয়েন্টার অপারেটর (& এবং *) এবং মেম্বার সিলেকশন অপারেটর (.এবং->)। এদের মধ্যে অন্যতম অপারেটর হলো কমা অপারেটর (,) ও সাইজ অফ অপারেটর (sizeof)।

কমা অপারেটর (Comma Operator) : একই ধরনের একাধিক এক্সপ্রেশনকে সংযুক্ত বা আলাদা করতে কমা অপারেটর ব্যবহৃত হয়। নিম্নে টেবিলের মাধ্যমে কমা অপারেটরের ব্যবহার দেখানো হলো–

এক্সপ্রেশন	কমা অপারেটর ব্যবহার করে সমতুল্য এক্সপ্রেশন
int x=5; int y=6; int z=7;	int x=5, y=6, z=7;

int i; int j = 10; for (i=1; i<=j; i+ +)	int i, j; for (i=1; i<=10; i++)

বিটওয়াইজ অপারেটর (Bitwise Operator)

সি প্রোগ্রামে বাইনারি ডেটা অর্থাৎ বিট/বাইট নিয়ে বিভিন্ন রকম যৌক্তিক অপারেশন যেমন- অর, এন্ড, নট, এক্স-অর, লেফ্‌ট শিফ্‌ট, রাইট শিফ্‌ট ইত্যাদি সম্পন্ন করার জন্য বিটওয়াইজ অপারেটর ব্যবহৃত হয়। এদের মধ্যে বিটওয়াইজ নট হলো ইউনারি অপারেটর, অন্যগুলো বাইনারি অপারেটর। নিম্নের টেবিলের মাধ্যমে বিটওয়াইজ অপারেটরসমূহের নাম ও ব্যবহার উল্লেখ করা হলো :

বিটওয়াইজ অপারেটর	ব্যবহার
| (বিটওয়াইজ অর)	দু'টি অপারেন্ডের বিটসমূহের মধ্যে জোড়ায় জোড়ায় OR অপারেশনের জন্য ব্যবহৃত হয়।
& (বিটওয়াইজ অ্যান্ড)	দু'টি অপারেন্ডের বিটসমূহের মধ্যে জোড়ায় জোড়ায় AND অপারেশনের জন্য ব্যবহৃত হয়।
^ (বিটওয়াইজ এক্স-অর)	দু'টি অপারেন্ডের বিটসমূহের মধ্যে জোড়ায় জোড়ায় X-OR অপারেশনের জন্য ব্যবহৃত হয়।
<< (বিটওয়াইজ লেফ্‌ট শিফ্‌ট)	কোনো অপারেন্ডের বাইনারি বিটসমূহকে এক বা একাধিক ঘর বামদিকে সরাতে ব্যবহৃত হয়।
>> (বিটওয়াইজ রাইট শিফ্‌ট)	কোনো অপারেন্ডের বাইনারি বিটসমূহকে এক বা একাধিক ঘর ডানদিকে সরাতে ব্যবহৃত হয়।
~ (বিটওয়াইজ নট)	কোনো অপারেন্ডের বিটগুলোকে বিপরীত (inverse) করার জন্য ব্যবহৃত হয়।

উল্লেখ্য, বিটওয়াইজ অপারেটর কেবল int টাইপের ডেটা নিয়ে কাজ করে। বিটওয়াইজ অপারেশনের পূর্বে কম্পাইলার উভয় অপারেন্ডের মান বাইনারিতে পরিবর্তন করে এবং অপারেন্ড দুইটির ডান দিক থেকে জোড়ায় জোড়ায় (বিট বাই বিট) অপারেশন সম্পন্ন করে।

সাধারণভাবে লজিক অপারেশনে আমরা জানি,

এন্ড গেইট X=AB

ইনপুট		আউটপুট
A	B	X=AB
0	0	0
0	1	0
1	0	0
1	1	1

OR গেইট X = A+B

ইনপুট		আউটপুট
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

X-OR গেইট

$X = A \oplus B = \bar{A}B + A\bar{B}$

ইনপুট		আউটপুট
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

বিটওয়াইজ . . .

p	q	p&q এন্ড গেইট	p|q অর গেইট	p^q X-OR গেইট
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

ধরা যাক, A = 60; এবং B = 13; এর বাইনারি মান হলো : A = 0011 1100 B = 0000 1101

সুতরাং , A&B = 0000 1100 হবে। এ মানটি যেভাবে হলো :

A = 0 0 1 1 1 1 0 0

B = 0 0 0 0 1 1 0 1

0 0 0 0 1 1 0 0

এখানে A এর মান 0 এবং B এর মান 1। সুতরাং A&B অর্থাৎ = A.B=0.1 = 0

এখানে A এর মান 1 এবং B এর মান 1। সুতরাং A&B অর্থাৎ = A.B =1.1= 1

A | B = 0011 1101 A ^ B = 0011 0001

লেফ্‌ট শিফ্‌ট অপারেশনের পূর্বে কম্পাইলার << এর বাম দিকের অপারেন্ডের মান বাইনারিতে পরিবর্তন করে। অতঃপর ডান দিকের অপারেন্ডের মান অনুযায়ী বিটগুলো এক এক করে বামদিকে সরিয়ে ডানদিকের ফাঁকা স্থান শূন্য দ্বারা পূর্ণ করে। অর্থাৎ, প্রতিবারে লেফ্‌ট শিফ্‌ট অপারেশনে কোন সংখ্যার মান দ্বিগুণ হয়। বিপরীতক্রমে, রাইট শিফ্‌ট অপারেশনের পূর্বে কম্পাইলার >> এর বাম দিকের অপারেন্ডের মান বাইনারিতে পরিবর্তন করে, অতঃপর ডান দিকের অপারেন্ডের মান

অনুযায়ী বিটগুলো এক এক করে ডানদিকে সরিয়ে বাম দিকের ফাঁকা স্থান শূন্য (0) দ্বারা পূর্ণ করে। অর্থাৎ প্রতিবার রাইট শিফট অপারেশনে কোন সংখ্যার মান অর্ধেক হয়।

অপারেন্ড	লেফট শিফট অপারেশনের মান			রাইট শিফট অপারেশনের মান		
	১ম শিফ্‌ট	২য় শিফ্‌ট	৩য় শিফ্‌ট	১ম শিফ্‌ট	২য় শিফ্‌ট	৩য় শিফ্‌ট
a = 2	a<<1=4	a<<2=8	a<<3=16	a>>1=1	a>>2=0	a>>3=0
b = 4	b<<1=8	b<<2=16	b<<3=32	b>>1=2	b>>2=1	b>>3=0

টেবিল : কয়েকটি লেফ্‌ট শিফট ও রাইট শিফ্‌ট অপারেশনের ফলাফল

অ্যাসাইনমেন্ট অপারেটর (Assignment Operator)

সি প্রোগ্রামে একটি ভেরিয়েবল বা এক্সপ্রেশনের মান অন্য কোনো ভেরিয়েবল বা এক্সপ্রেশনের মান হিসেবে নির্ধারণ করতে যে অপারেটর ব্যবহার করা হয়, তাকে অ্যাসাইনমেন্ট অপারেটর বলা হয়। সাধারণত অ্যাসাইনমেন্ট অপারেটর হিসেবে '=' চিহ্ন ব্যবহৃত হয়। যেমন : x=5, factorial = 0। এখানে x ও factorial হলো ভেরিয়েবলের নাম, যাদেরকে যথাক্রমে 5 ও 0 দিয়ে অ্যাসাইন করা হয়েছে।

x = x + 1	x + = 1
x = x – 1	x – = 1
x = x * y	x * = y
x = x / y	x / = y
x = x % y	x % = y

৮.৮ এক্সপ্রেশন (Expression)

কতগুলো অপারেন্ড, অপারেটর ও কনস্ট্যান্টের অর্থবোধক ও সামঞ্জস্যপূর্ণ উপস্থাপনকে এক্সপ্রেশন বা বর্ণনা বলা হয়। উদাহরণ হিসেবে বলা যায়, Avg = (V1 + V2)/2; একটি এক্সপ্রেশন। এখানে Avg, V1, V2 ও 2 অপারেন্ড এবং / = + হলো অপারেটর। কতগুলো অপারেন্ড, কনস্ট্যান্ট ও অ্যারিথমেটিক অপারেটরের মধ্যে সামঞ্জস্যপূর্ণ উপস্থাপন বা সম্পর্ককে অ্যারিথমেটিক বা গাণিতিক এক্সপ্রেশন বলা হয়।

উদাহরণ : Avg = (V1+V2)/2; একটি গাণিতিক এক্সপ্রেশন। এখানে Avg, V1, V2 অপারেন্ড, 2 কন্সট্যান্ট এবং =, +, / গাণিতিক অপারেটর।

সি-তে গাণিতিক ফর্মুলাকে সমতুল্য এক্সপ্রেশনে রূপান্তরের পদ্ধতি

সি-তে গাণিতিক ফর্মুলাকে সমতুল্য এক্সপ্রেশনে রূপান্তরের ক্ষেত্রে বিশেষ কতগুলো নিয়ম ব্যবহৃত হয়। যেমন– a^2 কে a* a হিসেবে লেখা হয়, 2ab কে 2*a*b হিসেবে লেখা হয়, $x \le y$ কে x <= y হিসেবে লেখা হয়, $x \neq y$ কে x != y হিসেবে লেখা হয় ইত্যাদি। নিম্নের ছকে গাণিতিক এক্সপ্রেশন এবং তাদের সমতুল্য সি এক্সপ্রেশন দেয়া হলো :

গাণিতিক এক্সপ্রেশন	সমতুল্য সি এক্সপ্রেশন
$x = a^2 + 2ab + b^2$	x = a*a + 2*a*b + b*b ;
$y = ax^2 + bx + c$	y = a*x*x + b*x + c ;
$d = \sqrt{(b^2 - 4ac)/2a}$	d = sqrt(b*b–4*a*c)/(2*a);
$\sqrt{(3a^2 + 6b^2)}$	sqrt(3*a*a + 6*b*b)
d = x ÷ y	d = x/y;
$\sqrt{81}$	Sqrt(81)
$\sqrt{s(s-a)(s-b)(s-c)}$	sqrt (s*(s – a)*(s – b)*(s – c)
$\sqrt{(b^2 - 4ac)/2a}$	sqrt (b*b – 4*a*c)/2*a
r = x mod y	r = x%y;
for(x = y)	for(x == y)
for(x ≠ y)	for(x != y)

গাণিতিক এক্সপ্রেশন	সমতুল্য সি এক্সপ্রেশন
for($x \le y$)	for(x <= y)
for($x \ge y$)	for(x >= y)
for x =1 to 10, x=x+1	for(x=1; x<=10; x=x+1)
for x =10 to 1 ,x=x-1	for(x=10; x>=1; x=x-1)
Area=$\pi r^2 + 2\pi rh$	Area=π*r*r+2*π*r*h
$y = x^4 + p/q$	y = pow(x,4)+p/q
$x = y^7 + z^2$	x = pow (y, 7) + z * z

উদাহরণ–১ : $a^2+2ab+b^2$ সমীকরণকে সি এক্সপ্রেশনে রূপান্তর

$a^2+2ab+b^2$= a * a + 2 * a * b + b * b

উদাহরণ–২ : Area = $\pi r^2 + 2\pi rh$ সমীকরণটিকে গাণিতিক ও সি এক্সপ্রেশনে রূপান্তর

Area = $\pi r^2 + 2\pi rh$ বা, Area = π* r * r + 2 * π * r* h → সি এক্সপ্রেশন

Area = π× r × r + 2 × π × r× h → গাণিতিক এক্সপ্রেশন

"=" এবং "= =" এর মধ্যে পার্থক্য

"=" হচ্ছে অ্যাসাইনমেন্ট অপারেটর। কোনো চলকের মানকে বা কোনো সংখ্যাকে অন্য কোনো চলকের মান হিসেবে নির্ধারণ করতে = অ্যাসাইনমেন্ট অপারেটর ব্যবহার করা হয়। যেমন- X=A+B। এখানে A ও B চলকের মান যোগ করে যোগফল X চলকে অ্যাসাইন করা হয়েছে। "= =" হলো রিলেশনাল অপারেটর। দুটি অপারেন্ডের মধ্যে তুলনা করতে রিলেশনাল অপারেটরসমূহ ব্যবহৃত হয়। যেমন : a==b হলো a ও b এর মান সমান।

উদাহরন-৩ : কয়েকটি গাণিতিক এক্সপ্রেশনের সি এক্সপ্রেশন :

গাণিতিক এক্সপ্রেশন	সি এক্সপ্রেশন
ক. $(a+b)^2$	ক. (a+b)*(a +b)
খ. $a^2–b^2$	খ. a*a–b* b
গ. (a+b) (a–b)	গ. (a+b)*(a–b)
ঘ. a^3+b^3	ঘ. a*a*a+b*b*b
ঙ. $a^3–b^3–3ab(a – b)$	ঙ. a*a*a–b*b*b–3*a*b*(a–b)
চ. $(a+b)^2–(a–b)^2$	চ. (a+b)*(a+b)–(a–b)*(a–b)

উদাহরণ-৪

(ক) 81/9/9=? (খ) 4*(3+8)=? (গ) 3*3*3=?

(ঘ) 3*2*2/4+3=? (ঙ) 36/(2*(3*3))=? (চ) 4*2*2/8=?

(ছ) 6/2+7-3*3=?

উত্তর : (ক) 1 (খ) 44 (গ) 27 (ঘ) 6 (ঙ) 2 (চ) 2 (ছ) 1

উদাহরণ-৫ : Pow((6/2+4/2),(13 MOD 5))–4*2+7 এর মান নির্ণয় :

Pow((6/2+4/2),(13 MOD 5))-4*2+7

= Pow((3+2),(3))-8+7

প্রোগ্রাম-৯ : এক্সপ্রেশনের ব্যবহার

a ও b এর মান নিয়ে $a^2+2ab+b^2$এর মান নির্ণয়

```
#include<stdio.h>
int main()
{
int a,b,f;
printf("Value of a=");
scanf("%d",&a);
printf("Value of b=");
scanf("%d",&b);
f = = a*a+2*a*b+b*b;
printf("Value of formula =%d",f);
return 0;
}
```

আউটপুট :

```
Value of a= 3
Value of b= 4
Value of formula =49
```

pow() হলো পাওয়ার ফাংশন।

pow(4,2)=4^2=4×4=16

pow(4,3)=4^3=4×4×4=64

pow(4,4)=4^4=4×4×4×4=256

mod( ) হলো মডিউলাস অর্থাৎ ভাগশেষ।

mod(14,3)=14÷3=2 (ভাগশেষ)

14÷4 = 4 | 14 | 3
12
2

= Pow((5,3)-1 = 5^3 - 1
= 125 - 1 = 124

উদাহরণ-৬ : Pow((15/3–9/3),(17 MOD 5))–4+3*2+7 এর মান নির্ণয় :

Pow((15/3–9/3),(17 MOD 5)) – 4 + 3*2 + 7
= Pow((5 – 3),(2)) – 4 + 6 + 7
= Pow((2,2) – 4 + 13 = 4 – 4 + 13 = 13

উদাহরণ-৭ : 49/7–Pow((18/3 –3/3), (14 MOD 4)) –5 + 2*2 + 29 এর মান নির্ণয় :

49/7 – Pow((18/3 –3/3), (14 MOD 4)) – 5 + 2*2 + 29
= 7– Pow((6 – 1),(2)) – 5+4+29 = 7– Pow((5, 2)–5+4 +29
= 7–25–5+33 = 40–30=10

কীওয়ার্ড (Keyword) বা সংরক্ষিত শব্দ

প্রত্যেক প্রোগ্রামিং ভাষার নিজস্ব কিছু সংরক্ষিত শব্দ আছে যা প্রোগ্রাম রচনার সময় ব্যবহার করা হয়। এই সংরক্ষিত শব্দগুলোকে কী-ওয়ার্ড বলা হয়। প্রত্যেকটি কীওয়ার্ডের একটি নির্দিষ্ট অর্থ আছে এবং প্রোগ্রামে একটি নির্দিষ্ট কার্য সম্পাদন করে। কীওয়ার্ডগুলো প্রচলিত অর্থের সাথে সামঞ্জস্যপূর্ণ নাও হতে পারে এবং কম্পাইলার যখন কোনো প্রোগ্রাম কম্পাইল করে তখন কীওয়ার্ড-এর জন্য কিছু পূর্বনির্ধারিত প্রোগ্রাম কোড উৎপন্ন করে। কীওয়ার্ডসমূহ ব্যবহারের জন্য সুনির্দিষ্ট নিয়ম আছে। এর সামান্য ব্যতিক্রম হলে প্রোগ্রাম ভুল ফলাফল দিতে পারে। নিম্নে কীওয়ার্ড ব্যবহারের কয়েকটি নিয়ম উল্লেখ করা হলো :

- কীওয়ার্ডসমূহের নাম একটি একক শব্দ বা ওয়ার্ডে হয়, অর্থাৎ মাঝে কোন ফাঁকা স্থান থাকে না।
- কীওয়ার্ড সব সময় ইংরেজি ছোট হাতের হয়।
- দু'টো কীওয়ার্ড একত্রে ব্যবহার করতে হলে মাঝে ফাঁকা স্থান হবে।

সি প্রোগ্রামে ব্যবহৃত কীওয়ার্ডসমূহ

বিভিন্ন কম্পাইলার বিভিন্ন কীওয়ার্ড সমর্থন করে। কিন্তু ANSI (American National Standard Institute) স্ট্যান্ডার্ড কম্পাইলার তাদের নিজস্ব কিছু কীওয়ার্ড ছাড়াও ANSI কীওয়ার্ড সমর্থন করে। সি প্রোগ্রামে ৩২টি স্ট্যান্ডার্ড কীওয়ার্ড রয়েছে। সি++ এ ৩২টি স্ট্যান্ডার্ড কীওয়ার্ড ছাড়াও নতুন আরো ৩০টি কীওয়ার্ড আছে, যা সি এ নেই। নিম্নের ছকে কীওয়ার্ডসমূহের তালিকা দেয়া হলো।

auto	double	int	struct
break	else	long	switch
case	enum	register	typeof
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

চিত্র : ৩২টি স্ট্যান্ডার্ড কীওয়ার্ড

asm	false	public	try
bool	friend	protected	typeid
catch	inline	reinterpret_cast	typename
class	mutable	static_cast	using
const_cast	namespace	template	virtual
delete	new	this	wchar_t
dynamic_cast	operator	throw	
explicit	private	true	

চিত্র : সি++ এ ৩২টি স্ট্যান্ডার্ড কীওয়ার্ড ছাড়াও নতুন আরো ৩০টি কীওয়ার্ড

ভেরিয়েবল ও কীওয়ার্ড-এর মধ্যে পার্থক্য

কীওয়ার্ড ব্যবহারের নিয়ম

- কীওয়ার্ড সবসময় ইংরেজি ছোট হাতের হবে।
- কীওয়ার্ড একটি একক শব্দ। এর অক্ষরের মাঝখানে কোনো ফাঁকা থাকবে না।
- কীওয়ার্ডকে ভেরিয়েবল (আইডেন্টিফায়ার) হিসেবে ব্যবহার করা যাবে না।

ভেরিয়েবল হলো প্রোগ্রামার কর্তৃক প্রদত্ত মেমোরি লোকেশনের নাম, যেখানে প্রোগ্রাম নির্বাহের সময় পরিবর্তনশীল মান বা ডেটা সংরক্ষণ করা হয়। কীওয়ার্ড হলো নির্দিষ্ট অর্থপূর্ণ কোনো ওয়ার্ড, যা প্রোগ্রামে একটি নির্দিষ্ট কার্য সম্পাদন করে। ভেরিয়েবলের নাম হিসেবে কোনো কীওয়ার্ড ব্যবহার করা যায় না।

কীওয়ার্ডকে ভেরিয়েবল হিসেবে ব্যবহার করা যায় না কেন?

প্রত্যেকটি কীওয়ার্ডের একটি নির্দিষ্ট অর্থ আছে ও প্রোগ্রামে একটি নির্দিষ্ট কার্য সম্পাদন করে এবং প্রোগ্রাম কম্পাইলের সময় প্রতিটি কীওয়ার্ড-এর জন্য পূর্বনির্ধারিত কিছু প্রোগ্রাম কোড উৎপন্ন হয়। এজন্য কীওয়ার্ডকে ভেরিয়েবল হিসেবে ব্যবহার করা যায় না।

স্টেটমেন্ট (Statement)

কম্পিউটার প্রোগ্রাম হলো কতগুলো নির্দেশের তালিকা যা কম্পিউটার দ্বারা নির্বাহ হয়। প্রোগ্রামিং ল্যাংগুয়েজে এসব নির্দেশের তালিকাসমূহকে স্টেটমেন্ট বলা হয়। যেমন- printf("Hello World!"); স্টেটমেন্টটি কম্পাইলারকে পর্দায় Hello World প্রদর্শন করার নির্দেশ দেয়। স্টেটমেন্টের শেষে সেমিকোলন দিতে হয়। সি প্রোগ্রামিং ল্যাংগুয়েজে মূলত: তিন ধরণের স্টেটমেন্টস রয়েছে। যথা-

১. এক্সপ্রেশন স্টেটমেন্টস (Expression Statements)
২. ডিক্লারেশন স্টেটমেন্টস (Declaration Statements)
৩. কন্ট্রোল ফ্লো স্টেটমেন্টস (Control Flow Statements)

নিচে উপরোক্ত স্টেটমেন্টসমূহের অধীন গুরুত্বপূর্ণ বিভিন্ন স্টেটমেন্ট সম্পর্কে আলোচনা করা হয়েছে।

ইনপুট স্টেটমেন্ট (Input Statement)

সি প্রোগ্রামে Standard I/O লাইব্রেরি ফাংশন রয়েছে, যা সব ধরনের ইনপুট/আউটপুটের কাজ সম্পন্ন করে। প্রোগ্রামে ডেটা তথা ভেরিয়েবলের মান গ্রহণ করার জন্য ব্যবহৃত স্টেটমেন্টকে ইনপুট স্টেটমেন্ট বলা হয়। কী-বোর্ড থেকে স্ট্রিং জাতীয় ডেটা ইনপুটের জন্য সি-তে কয়েক ধরনের স্টেটমেন্ট ব্যবহার করা হয়। যথা- scanf(), getc(), gets(), getchar()। সি ল্যাঙ্গুয়েজে তিন ধরনের ব্যবস্থার মাধ্যমে ডেটা ইনপুট দেয়া যেতে পারে। যথা-

১. অ্যাসাইনমেন্ট স্টেটমেন্ট (Assignment Statement)
২. ফরমেটেড ইনপুট (Formated Input)
৩. রিডিং এ ক্যারেক্টার (Reading a Character)

অ্যাসাইনমেন্ট স্টেটমেন্ট (Assignment Statement)

ডেটাকে সরাসরি ভেরিয়েবলে নির্দিষ্ট করে দেয়া হয়। যেমন- num1=76, num2= 84;

প্রোগ্রাম-১০: ২টি নির্দিষ্ট সংখ্যার যোগফল নির্ণয়

```
#include<stdio.h>
main()
{
int num1=76, num2=84, sum;
sum=num1+num2;
printf("summation of two number is =%d",sum);
return 0;
}
```

এখানে ভেরিয়েবল দুইটির মান নির্দিষ্ট (অ্যাসাইন) করে দেয়া হয়েছে। তাই এগুলোকে অ্যাসাইনমেন্ট স্টেটমেন্ট বলা হয়। দুটি ভেরিয়েবলে নির্দিষ্ট মানের যোগফল বের করার জন্য এরূপ প্রোগ্রাম রচনা করা যায়।

Output :

```
summation of two number is = 160
```

ইনপুট ফাংশন (Input Function)

scanf() একটি বহুল ব্যবহৃত ইনপুট স্টেটমেন্ট। প্রোগ্রাম রান করার সময় ব্যবহারকারীর নিকট থেকে প্রয়োজনীয় ডেটা নেয়ার জন্য এই ফাংশনটি ব্যবহৃত হয়। এর সাহায্যে int, char, float ইত্যাদি টাইপের ডেটা ইনপুট করা যায়। এর জন্য stdio.h লাইব্রেরি ফাংশন প্রোগ্রামে সংযুক্ত করতে হয়। নিম্নে scanf() ফাংশন-এর সিনট্যাক্স :

```
scanf ("Control String ", & Variable Name);
```

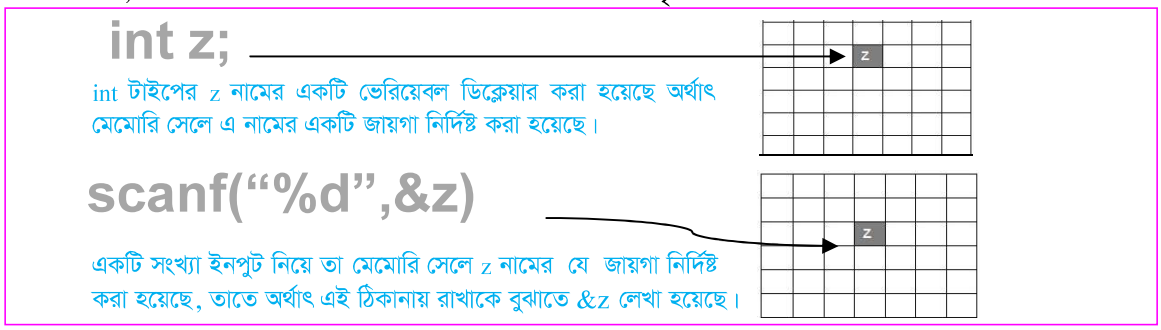
- Control String হলো ফরম্যাট স্পেসিফায়ার, যা কোন টাইপের ডেটা ইনপুট করবে তা নির্দেশ করে।
- Variable Name হলো পূর্বে ঘোষিত কোনো ভেরিয়েবলের নাম।
- & হলো অ্যাড্রেস অপারেটর, যা ভেরিয়েবলের জন্য গৃহীত মান মেমোরির ঐ লোকেশনে প্রেরণ করে এবং কোনো ভেরিয়েবলের মান বহন করে। যেমন :

```
int age;
scanf("%d", &age);
```

এখানে %d হলো integer ডেটা টাইপের ফরমেট স্পেসিফায়ার এবং &age হলো age নামের ভেরিয়েবলের অ্যাড্রেস, যা নির্দেশ করে integer টাইপের ডেটা age নামের ভেরিয়েবলের মেমোরি লোকেশনে সংরক্ষিত হবে।
দুই বা ততোধিক ইনপুট ডেটা নেয়ার জন্য স্টেটমেন্টের গঠন হলো :

```
scanf ("%d%d%d…%d", &variable1, &variable2, &variable3, . . . &variable n );
```

এখানে একই টাইপের ডেটার জন্য হতে পারে অথবা ভিন্ন টাইপের ডেটার জন্যও হতে পারে। ভেরিয়েবলসমূহ যদি একই টাইপের হয়, তাহলে আলাদা করে না লিখে একই লাইনে কমা দিয়ে পৃথক করে লেখা যায়।



ফরমেট স্পেসিফায়ার (Format Specier)

scanf() বা print() ফাংশন দ্বারা কোনো ডেটা বা ভেরিয়েবলের মান স্টোর বা প্রদর্শনের জন্য নির্দিষ্ট ফরম্যাটের কতগুলো ক্যারেক্টার ব্যবহার করা হয়, যেগুলোকে ফরমেট স্পেসিফায়ার বলা হয়। যেমন, %d ফরমেট স্পেসিফায়ারটি integer টাইপের ডেটা ইনপুট বা আউটপুট করার জন্য ব্যবহৃত হয়। নিচের টেবিলে বিভিন্ন ধরনের ফরমেট স্পেসিফায়ারের ব্যবহার উল্লেখ করা হলো :

স্পেসিফায়ার	ব্যবহার	উদাহরণ
%c	single character ইনপুট/আউটপুট করার জন্য ব্যবহৃত হয়।	scanf("%c",&name); printf("%c",name);
%d	integer টাইপের ডেটা ইনপুট/আউটপুট করার জন্য।	scanf("%d",&name); printf("%d",name);
%ld	long integer টাইপের ডেটা ইনপুট/আউটপুট করার জন্য।	scanf("%ld",&name); printf("%ld",name);
%f	floating point টাইপের ডেটা ইনপুট/আউটপুট করার জন্য।	scanf("%f",&name); printf("%f",name);
%lf	double টাইপের ডেটা ইনপুট/আউটপুট করার জন্য।	scanf("%lf",&name); printf("%lf",name);
%u	unsigned integer টাইপের ডেটা ইনপুট/আউটপুট করা।	scanf("%u",&name); printf("%u",name);
%lu	long unsigned integer ডেটা ইনপুট/আউটপুট করার জন্য।	scanf("%lu",&name); printf("%lu",name);

scanf () ফাংশনের ব্যবহার দেখার জন্য নিচের প্রোগ্রামটি লক্ষ করা যাক :

প্রোগ্রাম-১১: এন্ট্রি করা বিভিন্ন ধরনের মান প্রদর্শন

```
#include <stdio.h>
int main(){
char ch;
char str[100];
int num;
printf("Enter any character : \n");
scanf("%c", &ch);
printf("Entered character is %c \n", ch);
printf("Enter any string :\n");
scanf("%s", &str);
printf("Entered string is %s \n", str);
printf("Enter any number:\n", num);
scanf("%d", &num);
printf("Entered number is %d \n", num);
}
```

প্রোগ্রামটিতে char ও int এ দুই ধরনের ডেটা টাইপের ভেরিয়েবলে ক্যারেক্টার, স্ট্রিং ও নাম্বার টাইপের ডেটা ইনপুট করা হয়েছে। ক্যারেক্টারের জন্য %c, স্ট্রিং-এর জন্য %s এবং নাম্বার টাইপের জন্য %d ফরমেট স্পেসিফাইয়ার ব্যবহার করা হয়েছে।

Output :

```
Enter any character : a
Entered character is a
Enter any string : hasan
Entered string is hasan
Enter any number:342
Entered number is 342
```

আউটপুট স্টেটমেন্ট (Output Statement)

প্রোগ্রামে ডেটা তথা ভেরিয়েবলের মান প্রদর্শন করার জন্য ব্যবহৃত স্টেটমেন্টকে আউটপুট স্টেটমেন্ট বলা হয়। কী-বোর্ড থেকে স্ট্রিং জাতীয় ডেটা প্রদর্শনের জন্য সি-তে কয়েক ধরনের স্টেটমেন্ট ব্যবহার করা হয়। যেমন- printf(), putc(), puts(), putchar() ইত্যাদি।

printf () ফাংশন

এটি একটি বহুল ব্যবহৃত আউটপুট স্টেটমেন্ট। এর সাহায্যে int, char, float ইত্যাদি টাইপের ডেটার মান মনিটরের স্ক্রিনে প্রদর্শন করা যায়। printf () এর সিনট্যাক্স হলো-

```
printf ("string ");
```

String হিসেবে যে কোন ওয়ার্ড বা ক্যারেক্টার বা সেনটেন্স হতে পারে। printf এর প্রথম বন্ধনী () এর ভেতর ডাবল কোটেশনের মধ্যে যা লেখা হয়, আউটপুট হিসেবে মনিটরের পর্দায় তা প্রদর্শিত হয়। উপরের প্রোগ্রামটিতে scanf() এর পাশাপাশি printf() ফাংশনের ব্যবহার দেখানো হয়েছে। প্রোগ্রামের আউটপুট লক্ষ করলে আমরা দেখতে পাই printf() ফাংশনের ডাবল কোটেশনের ভিতর দিয়ে % সহ যে অক্ষর যাকে ফরমেট স্পেসিফাইয়ার বলা হয় তাতে উক্ত টাইপের ভেরিয়েবলের মান প্রদর্শিত হয়। যেমন-

```
printf("Entered number is %d\n", num);
```

এখানে int (integer) টাইপের num নামক ভেরিয়েবল-এর ভেলু (342) উল্লিখিত %d ফরমেট স্পেসিফাইয়ারে প্রদর্শিত হচ্ছে। অর্থাৎ এখানে %d ইন্টিজার (পূর্ণসংখ্যা) টাইপের num নামক ভেরিয়েবলের মান দ্বারা প্রতিস্থাপিত হচ্ছে।

প্রোগ্রাম-১২ : মি. আসিফের বয়স রাকিবের বয়সের পাঁচগুণ। বাবুর বয়স আসিফের বয়স অপেক্ষা ১০ বছর বেশি। তাদের তিনজনের বয়স একত্রে কত তা নির্ণয়ের জন্য সি ভাষায় প্রোগ্রাম।

```
# include <stdio.h>
main ( ){
int K,S,B,Total;
pintf ("Enter Tanussri age=)";                    ধরো, রাকিবের বয়স K বৎসর।
scanf("%d",&K);
S= 5*K;
B =S+10;
Total= K+S+B;
printf("The required sum is; %d", Total);
return 0;
}
```

প্রোগ্রাম-১৩ : পিতার বয়স পুত্রের বয়সের তিনগুণ। মায়ের বয়স পিতার বয়স অপেক্ষা 5 বছর কম। পুত্রের বয়স x বছর। তাদের তিনজনের বয়স একত্রে কত তা নির্ণয়ের জন্য সি ভাষায় প্রোগ্রাম।

```
# include <stdio.h>
main ( ){
int X,M,F,Total;
pintf ("Enter son age=)";
scanf("%d",&X);
F = 3*X;
M =3*X-5;
Total= X+M+F;
printf("The required sum is; %d", Total);
return 0;
}
```

কন্ট্রোল স্টেটমেন্ট (Control Statement)

সি-ল্যাঙ্গুয়েজে লেখা প্রোগ্রামে স্টেটমেন্টগুলো ধারাবাহিকভাবে সাজানো থাকে, যা একের পর এক সিকোয়েন্স অনুযায়ী নির্বাহ হয়। এক্ষেত্রে স্টেটমেন্টগুলোর পুনরাবৃত্তি অনুযায়ী নির্বাহের প্রয়োজন হয় না। কিন্তু স্টেটমেন্টগুলোর পুনরাবৃত্তি অনুযায়ী নির্বাহের প্রয়োজন হলে কিছু শর্তযুক্ত করা হয়। এগুলোকে কন্ট্রোল স্টেটমেন্ট বলা হয়। স্টেটমেন্ট নির্বাহ নিয়ন্ত্রণের জন্য কন্ট্রোল স্টেটমেন্ট ব্যবহৃত হয়। কন্ট্রোল স্টেটমেন্ট ব্যবহার করে প্রোগ্রাম লেখা হলে প্রোগ্রামের আকার ছোট হয় এবং প্রোগ্রাম নির্বাহের সময় কম লাগে। সি-ভাষায় কন্ট্রোল স্টেটমেন্টকে সাধারণত তিন ভাগে ভাগ করা যায় :

১. কন্ডিশনাল কন্ট্রোল স্টেটমেন্ট (Conditional Control Statement)
২. লুপ কন্ট্রোল স্টেটমেন্ট (Loop Control Statement)।
৩. জাম্পিং কন্ট্রোল স্টেটমেন্ট (Jumping Control Statement)

কন্ডিশনাল কন্ট্রোল স্টেটমেন্ট (Conditional Control Statement)

সি/সি++ প্রোগ্রামে শর্তসাপেক্ষে কোন স্টেটমেন্ট সম্পাদনের জন্য if, if...else, else if ও switch ইত্যাদি স্টেটমেন্ট ব্যবহৃত হয়। এদেরকে ডিসিশন মেকিং স্টেটমেন্ট (Decision Making Statement) বা কন্ডিশনাল কন্ট্রোল স্টেটমেন্ট বলা হয়। কন্ডিশনাল কন্ট্রোলে ব্যবহৃত শর্ত সত্য হলে প্রোগ্রামে এক ধরনের ফল পাওয়া যায়, আর সত্য না হলে অন্য ধরনের ফল পাওয়া যায়।

if স্টেটমেন্ট

সি/সি++ প্রোগ্রামে "যদি" অর্থে if স্টেটমেন্ট ব্যবহৃত হয়। প্রোগ্রামে শর্ত সাপেক্ষে কোন স্টেটমেন্ট সম্পাদনের জন্য if স্টেটমেন্ট ব্যবহৃত হয়। if স্টেটমেন্ট-এর ফরম্যাট হলো :

```
if(Condition)
 {
 Block1;
 }
Block2;
```

টোকেন : সি প্রোগ্রামিং-এর সবচাইতে ক্ষুদ্রতম একককে টোকেন বলে। যে কোনো প্রোগ্রাম কতকগুলো স্টেটমেন্ট নিয়ে গঠিত। আবার প্রতিটি স্টেটমেন্টে কতকগুলো word ও character এর সমষ্টি। C তে ব্যবহৃত এরূপ word বা character ও symbol সমূহকে একত্রে টোকেন বলে।

if স্টেটমেন্টে ব্যবহৃত শর্ত (Condition) সাধারণত এক বা একাধিক লজিক্যাল বা রিলেশনাল এক্সপ্রেশন হয়, যা if পরবর্তী প্রথম বন্ধনীর মধ্যে লেখা হয়। if (Condition) স্টেটমেন্টের পর কোনো সেমিকোলন থাকবে না। এ শর্তের মান যদি সত্য হয়, তবে Block1 সম্পাদিত হয়। অন্যথায় Block1 সম্পাদিত হয় না। তবে উভয় ক্ষেত্রে Block2 সহ পরবর্তী স্টেটমেন্টসমূহ সম্পাদিত হয়। Block1 এ যে কোনো বৈধ সিম্পল বা কম্পাউন্ড স্টেটমেন্ট হতে পারে। নিচে if স্টেটমেন্ট ব্যবহার করে প্রোগ্রাম তৈরি করে দেখানো হয়েছে :

প্রোগ্রাম-১৪ : পাস করা প্রদর্শন

```
# include<stdio.h>
int main() {
int num1;
printf("Enter your number :");
scanf("%d", &num1);
if(num1>=33)
printf("\n you pass in the examination");
return 0;
}
```

প্রোগ্রামটি চালু করলে একটি সংখ্যা চাইব। এন্ট্রি করা সংখ্যাটি num1 নামক ভেরিয়েবলে রাখবে। সংখ্যাটি যদি 33 এর সমান বা তার চেয়ে বেশি হয়, তাহলে বার্তা দেখাবে :

you pass in the examination

- সংখ্যাটি 33এর সমান বা তার চেয়ে বেশি হলে উপরিউক্ত বার্তা দেখানোর জন্য প্রোগ্রামে শর্তারোপ করা হয়েছে এভাবে–

```
if(num1>=33)
printf("\nyou pass the examination"
```

- বার্তাটি নতুন লাইনে হওয়ার জন্য \n ব্যবহার করা হয়েছে।

ফলাফল :

```
Enter your number: 44
you pass in the Examination
```

প্রোগ্রামটি রান করিয়ে 33 এর চেয়ে ছোট সংখ্যা এন্ট্রি করলে কোনো ফলাফল দেখাবে না। কিন্তু আমরা যদি চাই, 33 এর চেয়ে কম হলে বার্তা দেখাবে, তাহলে প্রোগ্রামে নিচের মতো দুই লাইন সংযুক্ত করতে হবে।

```
if(number<33)
printf("\nyou have not passed in the examination");
```

if স্টেটমেন্ট ব্যবহার করে প্রোগ্রামটি দেখা যাক :

```
// using if
#include<stdio.h>
int main(){
int num1, num2;
printf("Enter two numbers:");
scanf("%d %d",&num1,&num2);
if(num1 == num2)
printf("they are equal");
if(num1> num2)
printf("%d is greater than %d", num1, num2);
if(num1 <num2)
printf("%d is less than %d", num1, num2);
return 0;
}
```

প্রোগ্রাম ১৫: ২টি সংখ্যার মধ্যে বৃহত্তম সংখ্যা নির্ণয়

আউটপুট :

```
Enter two numbers: 84 56
84 is greater then 56
```

প্রোগ্রামটি চালু করলে দুইটি সংখ্যা চাইব। 84 লিখে স্পেসবার চেপে 56 লিখে এন্টার করা হলে প্রথম সংখ্যা 84, num1 এ রাখবে এবং দ্বিতীয় সংখ্যা 56, num2 এ রাখবে।

এবার প্রথম শর্তের সাথে মিলিয়ে দেখেছে সংখ্যা দুইটি এক নয় অর্থাৎ num1 == num2 নয়। আবার পরবর্তী শর্তের সাথে মিলিয়ে দেখেছে 84, 56 এর চেয়ে বড় অর্থাৎ num1> num2. তাই পর্দায় লেখা প্রদর্শিত হবে : 84 is greater than 56

নিজে কর :
এন্ট্রি করা দ্বিতীয় সংখ্যাটি বড় হলে অর্থাৎ Enter two numbers: 56 84 হলে বার্তা দেখাবে : 56 is less than 84.
এভাবে আউটপুট পাওয়ার জন্য প্রোগ্রামটিতে কি পরিবর্তন করতে হবে তা লেখ।

প্রোগ্রাম-১৬ : if স্টেটমেন্ট ব্যবহার করে কোন সংখ্যা ধনাত্মক নাকি ঋণাত্মক তা বের করার প্রোগ্রাম:

```
#include<stdio.h>
int main(){
int a;
printf("Insert a number:");
scanf("%d",&a);
if(a>0)
printf("The number is positive\n");
if(a<0)
printf("The number is negative.\n");
return 0;
}
```

প্রোগ্রামটি রান করালে একটি সংখ্যা চাইব। সংখ্যাটি যদি 0 এর চেয়ে বড় হয়, তাহলে সংখ্যাটি পজেটিভ, সংখ্যাটি যদি 0 এর চেয়ে ছোট হয়, তাহলে সংখ্যাটি নেগেটিভ বার্তা দেখাবে।

আউটপুট :

```
Insert a number: 7
The number is positive.
```

নিজে কর : প্রোগ্রামটি রান করালে একটি সংখ্যা চাইব। সংখ্যাটি যদি 0 হয় তাহলে The number is zero এই বার্তা দেখাবে।
হিন্ট্স : প্রোগ্রামটিতে নিচের লাইনটি যুক্ত করতে হবে-

```
if(a==0)
printf("The number is zero.\n");
```

প্রোগ্রাম-১৭ : বয়স অনুসারে বার্তা প্রদর্শন করা।

```
#include<stdio.h>
int main() {
int Age;
printf("Please enter your age: ");
scanf("%d", &Age);
if(Age>0 && Age<=12)
 printf("\nYou are a child.");
if(Age>12 && Age<=19)
 printf("\nYou are a teen ager.");
return 0;
}
```

প্রোগ্রামটি রান করালে Please enter your age: বার্তা দেখিয়ে বয়স চাইব। বয়স 0-12 হলে You are a child বার্তা দেখাবে। বয়স 12 এর বেশি, কিন্তু 19 এর কম বা সমান হলে You are a teen ager বার্তা দেখাবে।

আউটপুট :

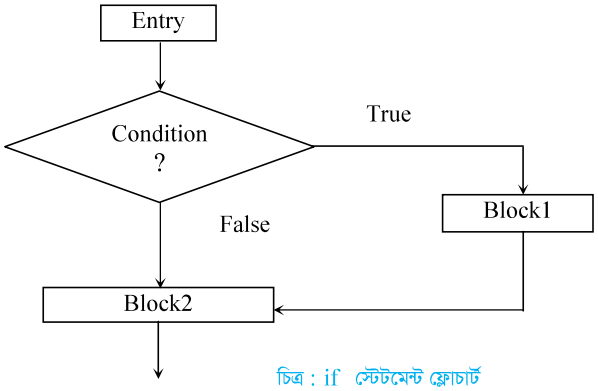
```
Please enter your age: 7
You are a child.
```

নিজে কর : বয়স 19 এর বেশি, কিন্তু 40 এর কম বা সমান হলে You are young বার্তা দেখাবে এবং বয়স 40 এর বেশি হলে Wish your long life বার্তা দেখাবে; এরূপ করার জন্য প্রোগ্রামটি মডিফাই করে রান করাও।

if ... else স্টেটমেন্ট

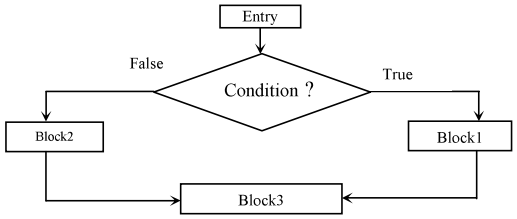
সি/সি++ প্রোগ্রামে 'অন্যথায়' অর্থে if স্টেটমেন্টের সাথে else স্টেটমেন্ট ব্যবহৃত হয়। এজন্য এ স্টেটমেন্টকে if...else স্টেটমেন্ট বলা হয়। if... else স্টেটমেন্ট ব্যবহারের ফরম্যাট হলো :

```
if(Condition)
 {
  Block1;
 }
else{
 Block2;
 }
Block3;
```



চিত্র : if স্টেটমেন্ট ফ্লোচার্ট

else কন্ট্রোলে ব্যবহৃত শর্ত (Condition) সাধারণত এক বা একাধিক লজিক্যাল বা রিলেশনাল এক্সপ্রেশন হয়, যা if এর পরে প্রথম বন্ধনীর মধ্যে লেখা হয়। if স্টেটমেন্টের পরে কোনো সেমিকোলন বসে না। এ শর্তের মান যদি সত্য হয়, তবে Block1 সম্পাদিত হয়। অন্যথায় Block1 সম্পাদিত না হয়ে Block2 সহ পরবর্তী স্টেটমেন্ট সম্পাদিত হয়। Block1 ও Block2 যে কোন বৈধ সিম্পল বা কম্পাউন্ড স্টেটমেন্ট হতে পারে। চিত্রে if..else স্টেটমেন্টের প্রবাহচিত্র দেখানো হলো :



চিত্র : if...else স্টেটমেন্টের প্রবাহচিত্র

if	যদি
if else	অন্যথায়
else if	অন্যথায় যদি

একটি if...else স্টেটমেন্টের মধ্যে অপর একটি if...else স্টেটমেন্টও থাকতে পারে। এরূপ মধ্যবর্তী if.. else স্টেটমেন্টকে নেস্টেড if...else স্টেটমেন্ট বলা হয়।

প্রোগ্রাম-১৮ : জোড় বা বেজোড় সংখ্যা নির্ণয়।

```
#include<stdio.h>
int main(){
   int n;
   printf("Enter a number :");
   scanf("%d",&n);
   if(n%2 ==0)
     printf("This is even number");
     else
      printf("This is odd number");
   return 0;
}
```

প্রোগ্রামটি চালু করলে একটি সংখ্যা চাইব। এন্ট্রি করা সংখ্যাটি n ভেরিয়েবলে রাখবে। সংখ্যাটি যদি জোড় হয় অর্থাৎ একে 2 দিয়ে ভাগ করলে যদি ভাগফল নিঃশেষে বিভাজ্য হয়, তাহলে সংখ্যাটি even (জোড়) হবে। আর যদি তা না হয় অর্থাৎ সংখ্যাটিকে 2 দিয়ে ভাগ করলে ভাগফল যদি নিঃশেষে বিভাজ্য না হয়, তাহলে সংখ্যাটি odd (বেজোড়) । সি প্রোগ্রামে ভাগ করার জন্য % চিহ্ন এবং মান সমান বুঝার জন্য == চিহ্ন ব্যবহার করা হয়েছে।

আউটপুট : Enter a number:7
This is odd number.

else if স্টেটমেন্ট

একাধিক শর্ত যাচাইয়ে সি/সি++ প্রোগ্রামে 'অন্যথায় যদি' অর্থে if...else স্টেটমেন্টের সাথে else if স্টেটমেন্ট ব্যবহৃত হয়। else if স্টেটমেন্ট if...else স্টেটমেন্টের if ও else স্টেটমেন্টের মাঝে বসে। else if স্টেটমেন্টে else ও if এর মাঝে ফাঁকা স্থান থাকে। প্রোগ্রামে একাধিক if স্টেটমেন্ট ব্যবহারের বিকল্প হিসেবে else if স্টেটমেন্ট জনপ্রিয়। অর্থাৎ প্রোগ্রামে একাধিক শর্ত যাচাই করার জন্য else if ব্যবহৃত হয়।

প্রোগ্রাম-১৯ : দুইটি পূর্ণসংখ্যার মধ্যে বড় সংখ্যাটি নির্ণয় করে ফলাফল প্রদর্শন।

```
#include <stdio.h>
main(){
int x, y;
printf("Enter two integer numbers: ");
scanf("%d %d", &x, &y);
if(x>y)
printf("The number %d is greater than %d", x, y);
else if (x<y)
printf("The number %d is greater than %d", y, x);
else
printf("The number %d is equal to %d", x, y);
}
```

- প্রোগ্রামটি চালু করলে দুইটি সংখ্যা চাইব। এন্ট্রি করা প্রথম সংখ্যাটি x ভেরিয়েবলে এবং দ্বিতীয় সংখ্যাটি y ভেরিয়েবলে রাখবে।
- ইনপুটকৃত সংখ্যা দুইটির মধ্যে প্রথম (x) টি যদি অপর (y)-টি থেকে বড় হয়, তাহলে if স্টেটমেন্ট-এর ব্লক নির্বাহ হবে এবং বার্তা দেখাবে :
  The number is (১ম সংখ্যা) greater than (২য় সংখ্যা)
- অন্যথায় else if এর ব্লক নির্বাহ হবে এবং বার্তা দেখাবে-
  The number is (২য় সংখ্যা) greater than (১ম সংখ্যা)
- উপরের কোনোটি সত্য না হলে, তারা সমান। অর্থাৎ else এর ব্লক নির্বাহ হবে।

আউটপুট :

```
Enter two integer numbers: 23 45 ↵
The number 45 is greater than 23
```

প্রোগ্রাম-২০ : ইনপুট করা অক্ষরটি small লেটার নাকি capital লেটার তা বের করা।

```
#include<stdio.h>
int main(){
char ch;
printf("Enter a character : ");
scanf("%c", &ch);
if((ch>= 'A') && (ch <= 'Z'))
printf("\nYou entered a capital letter : %c", ch);
else if((ch >= 'a') && (ch <= 'z'))
printf("\nYou entered a small letter : %c", ch);
else
printf("\nThe letter you entered is not a character.");
return 0;
}
```

প্রোগ্রামটি চালু করলে একটি অক্ষর চাইব। এন্ট্রি করা অক্ষরটি **যদি** ইংরেজি বড় হাতের (ক্যাপিটাল লেটার) হয়, তাহলে You entered a capital letter বার্তা দেখাবে **অথবা যদি** এন্ট্রি করা অক্ষরটি ইংরেজি ছোট হাতের (স্মল লেটার) হয়, তাহলে You entered a small letter বার্তা দেখাবে। যদি এন্ট্রি করা অক্ষরটি অন্য কোন ক্যারেক্টার হয়, তাহলে The letter you entered is not a character বার্তা দেখাবে

আউটপুট :

```
Enter a character:
R ↵
You entered a capital letter R
```

প্রোগ্রাম-২১ : if, if…else, if…elseif….else এর তুলনামূলক ব্যবহার : প্রোগ্রাম তিনটি সংখ্যা থেকে বড় সংখ্যাটি প্রদর্শন

if

```
#include <stdio.h>
int main(){
int x, y, z;
printf("Input three integer numbers: ");
scanf("%d%d%d",&x,&y,&z);
if((x>y)&&(x>z))
printf("%d is the largest number",x);
if((y>x)&&(y>z))
printf("%d is the largest number",y);
if((z>x)&&(z>y))
printf("%d is the largest number",z);
return 0;
}
```

আউটপুট :

```
Input three integer numbers:
12  32  17
 32 is the largest number
```

if…else

```
#include <stdio.h>
int main(){
int x, y, z;
printf("Input three integer numbers: ");
scanf("%d%d%d",&x,&y,&z);
if((x>y)&&(x>z))
printf("%d is the largest number",x);
if((y>x)&&(y>z))
printf("%d is the largest number",y);
else
printf("%d is the largest number",z);
return 0;
}
```

আউটপুট :

```
Input three integer numbers:
12  32  17
 32 is the largest number
```

If…else if….else

```
#include <stdio.h>
int main(){
int x, y, z;
printf("Input three integer numbers: ");
scanf("%d%d%d",&x,&y,&z);
if((x>y)&&(x>z))
printf("%d is the largest number",x);
else if((y>x)&&(y>z))
printf("%d is the largest number",y);
else
printf("%d is the largest number",z);
return 0;
}
```

আউটপুট :

```
Input three integer numbers:
12  32  17
 32 is the largest number
```

প্রোগ্রাম-২২ : স্বরবর্ণ নাকি ব্যঞ্জনবর্ণ তা বের করা

```
#include<stdio.h>
int main()
{
char ch;
printf("Enter a character: ");
scanf("%c", &ch);

if(ch=='A'|| ch=='a'||ch=='E'||

ch=='e'||ch=='I'|| ch=='i'||ch=='O'
|| ch=='o'||ch=='U'|| ch=='u')

printf("\n %c is a vowel", ch);
else
printf("\n %c is not vowel", ch);
}
```

Output:
Enter a character: U ↵
U is a vowel.

প্রোগ্রাম-২৩ : vowel বের করা

```
#include<stdio.h>
void main() {
char ch;
printf("Enter a character: ");
scanf("%c", &ch);
switch(ch){
case 'a':
case 'A':
case 'e':
case 'E':
case 'i':
case 'I':
case 'o':
case 'O':
case 'u':
case 'U':
printf("\n %c is a vowel", ch);
break;
default:
printf("\n %c is not vowel", ch);
}
}
```

Output:
Enter a character: s ↵
s is not vowel.

প্রোগ্রাম-২৪ : কোন সংখ্যা ইনপুট দিলে তা যদি 5 দ্বারা বিভাজ্য হয় তাহলে Flower শব্দটি প্রদর্শিত হয় এবং ৭ দ্বারা বিভাজ্য হলে River শব্দটি প্রদর্শিত হয়। কিন্তু 5 ও 7 উভয় দ্বারা বিভাজ্য হলে good শব্দটি প্রদর্শিত হয়। আর 5 ও 7 কোনটিই দ্বারা বিভাজ্য না হলে Try again শব্দটি প্রদর্শিত হয়।

```
#include <stdio.h>
int main() {
   int number;
   printf("Enter a number:");
   scanf("%d", &number);
   if (number % 5 == 0 && number % 7 == 0)
    {
      printf("good\n");
   }
   else if (number % 5 == 0)
   {
      printf("Flower\n");
   }
   else if (number % 7 == 0)
   {
      printf("River\n");
   }
   else {
      printf("Try again\n");
   }
   return 0;
}
```

Output:
Enter a number:35 ↵
good

প্রোগ্রাম-২৫ : বার্ষিক ক্রীড়া প্রতিযোগিতায় একাদশ শ্রেণির শিক্ষার্থীদের A, B ও C দলে বিভক্ত করা হয়। রোল নম্বর 1 থেকে 30 পর্যন্ত A দলে, 31 থেকে 60 পর্যন্ত B দলে এবং 61 থেকে 100 পর্যন্ত C দলে অন্তর্ভুক্ত হবে। সি ভাষায় কন্ডিশনাল স্টেটমেন্ট ব্যবহার করে দল গঠনের জন্য একটি প্রোগ্রাম রচনা কর।

```
#include <stdio.h>
int main() {
   int roll;
   printf("Enter the student's roll number(1-100): ");
   scanf("%d", &roll);
   if (roll >= 1 && roll <= 30) {
      printf("This student is included in group 'A'.\n");
  }
   else if (roll >= 31 && roll <= 60) {
      printf("This student is included in group 'B'.\n");
   }
   else if (roll >= 61 && roll <= 100) {
      printf("This student is included in group 'C'.\n");
   }
   else {
      printf("Invalid roll number! Please enter a number between 1 and 100.\n");
   }
   return 0;
}
```

Output:
□□□□□□□□□□□ □□□ □□□□□ □□□□□ (1-100):
45↵

প্রোগ্রাম-২৬ : পরীক্ষার ফলাফল নির্ণয় কর যেখানে 80-100=A+, 70-79=A, 60-69=A-,50-59=B,40-49=C,33-39=D, তার থেকে কম নম্বর পেলে F দেখাবে।

```
#include <stdio.h>
int main() {
    int marks;
    printf("Enter your obtained marks(0-100):");
    scanf("%d", &marks);

    if (marks >= 80 && marks <= 100) {
        printf("Grade: A+\n");
    }
    else if (marks >= 70 && marks <= 79) {
        printf("Grade: A\n");
    }
    else if (marks >= 60 && marks <= 69) {
        printf("Grade: A-\n");
    }
    else if (marks >= 50 && marks <= 59) {
        printf("Grade: B\n");
    }
    else if (marks >= 40 && marks <= 49) {
        printf("Grade: C\n");
    }
    else if (marks >= 33 && marks <= 39) {
        printf("Grade: D\n");
    }
    else if (marks >= 0 && marks < 33) {
        printf("Grade: F\n");
    }
    else {
        printf("Invalid input! Please enter a number between 0 and 100.\n");
    }
   return 0;
}
```

Output:

Enter your obtained marks(0-100):82

Grade: A+

লুপিং স্ট্রাকচার (Looping Structure)

প্রোগ্রামে কোনো স্টেটমেন্ট দুই বা ততোধিকবার সম্পাদিত হওয়ার প্রয়োজন হয়। যেমন নিচের প্রোগ্রামটি দেখা যাক :

প্রোগ্রাম-২৭ : কোন টেক্সটকে একাধিকবার প্রদর্শন করার প্রোগ্রাম

```
#include<stdio.h>
int main(){
int i;
for (i=1;i<=7;i++)
printf("\n I love Bangladesh");
return 0;
}
```

```
#include<stdio.h>
int main (){
int i=1;
while(i<=7)
{
printf("\n I love Bangladesh");
i = i+1;
}
}
```

```
#include<stdio.h>
int main (){
int i=1;
do {
printf("\n I love Bangladesh");
i = i+1;
} while (i<=7);
}
```

উক্ত তিনটি প্রোগ্রাম একই কাজ করবে অর্থাৎ সাতবার I love Bangladesh লেখাটি পর্দায় প্রদর্শন করবে। এখানে একই স্টেটমেন্ট সাতবার সম্পাদিত হয়েছে। প্রোগ্রামে যে সব স্টেটমেন্ট দুই বা ততোধিকবার সম্পাদিত হয়, সেগুলোকে লুপিং স্টেটমেন্ট বলা হয়। প্রোগ্রামে স্টেটমেন্টসমূহ সাধারণত স্বয়ংক্রিয়ভাবে ও পর্যায়ক্রমে একবার করে সম্পাদিত হয়। কিন্তু যদি কোনো স্টেটমেন্ট দুই বা ততোধিকবার সম্পাদনের প্রয়োজন হয়, সেসব ক্ষেত্রে লুপিং স্টেটমেন্ট ব্যবহৃত হয়। সি প্রোগ্রামে ব্যবহৃত লুপ স্টেটমেন্টগুলো হলো-for, while , do...while, continue ও goto স্টেটমেন্ট।

for লুপ স্টেটমেন্ট

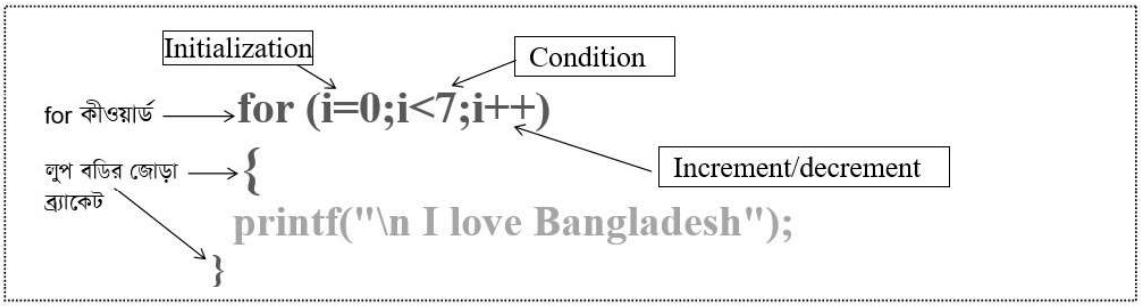
সি-তে কোনো স্টেটমেন্ট দুই বা ততোধিক বার সম্পাদন করার জন্য for স্টেটমেন্ট ব্যবহৃত হয়। সাধারণ কোনো ভেরিয়েবল ব্যবহার করে for লুপের আবর্তন সংখ্যা গণনা করা হয়। এরূপ ভেরিয়েবলকে কাউন্টার ভেরিয়েবল বলা হয়। কাউন্টার ভেরিয়েবলকে for স্টেটমেন্টে ব্যবহারের পূর্বেই ঘোষণা করতে হয়।

```
Counter Declaration;
for(Initialization; Condition; CounterUpdate)
{
LoopBody;
}
```

প্রোগ্রাম-২৮ : লেখা নির্দিষ্ট সংখ্যক বার দেখানো

```
#include<stdio.h>
int main()
{
int i;
for (i=0;i<7;i++)
{
printf("\n I love Bangladesh");
}
return 0;
}
```

কম্পাইলার প্রথমে for লুপের Initialization অংশে আসবে এবং i এর মান 0 পাবে, তারপর Condition অংশে দেখবে i এর মান 0 হলে Condition সত্য কিনা? সত্য; কারণ 0 হলো 7 এর নিচে। তাই for লুপের মধ্যে ঢুকবে অর্থাৎ for লুপের নিচে যে স্টেটমেন্ট আছে, সেটি কার্যকর করবে। তারপর ইনক্রিমেন্ট অংশে আসবে। i এর মান 1 করে বাড়বে। অর্থাৎ i এর মান হবে 0+1=1। এবার আবার আগের মতো Condition অংশে দেখবে i এর মান 1 হলে Condition সত্য কিনা? সত্য, কারণ 1 হলো 7 এর নিচে। তাই for লুপের মধ্যে ঢুকবে অর্থাৎ for লুপের নিচে যে স্টেটমেন্ট আছে, সেটি কার্যকর করবে। এভাবে যতক্ষণ Condition সত্য হবে, ততক্ষণ for লুপের নিচে যে স্টেটমেন্ট আছে, তা কার্যকর করবে অর্থাৎ ৭ বার I love Bangladesh প্রদর্শন করবে। যদি for লুপে i = 0 না ধরে i = 1 ধরা হতো, তাহলে I love Bangladesh লেখাটি ৬ বার দেখাতো।



এখানে প্রোগ্রামে যত সংখ্যা উল্লেখ আছে, তত সংখ্যক বার লেখা (টেক্সট) প্রদর্শিত হয়েছে। কিন্তু প্রোগ্রামটি রান করিয়ে টেক্সট এন্ট্রি করার পর যত সংখ্যা ইনপুট করা হবে, তত সংখ্যক বার টেক্সটটি প্রদর্শিত হবে; এরূপ চাইলে নিচের মতো প্রোগ্রাম লিখতে হবে।

প্রোগ্রাম-২৯: নির্দিষ্ট সংখ্যক কোনো লেখা প্রদর্শন করা।

```
#include<stdio.h>
int main(){
int i, n;
char txt[50];
printf("Enter the txt: ");
scanf("%[^\n]", &txt);
printf("How many times display ? ");
scanf("%d", &n);
for (i=1; i<=n; i++)
{
printf("%s \n", txt);
}
```

- যে লেখাটি প্রদর্শন করতে হবে, সেটি এন্ট্রি করা হলে ক্যারেক্টার টাইপের ভেরিয়েবলে রাখার জন্য char txt[50] ডিক্লেয়ার করা হয়েছে। এখানে [50] দিয়ে লেখার পরিমাণ অর্থাৎ ক্যারেক্টার সংখ্যা বুঝানো হয়েছে।
- স্পেসসহ লেখাকে প্রদর্শন করার জন্য %[^\n]", এই Advanced format sequence ব্যবহার করা হয়েছে।

Output:

```
Enter the text : dhaka ↵
How many times display ? 3 ↵
dhaka
dhaka
dhaka
```

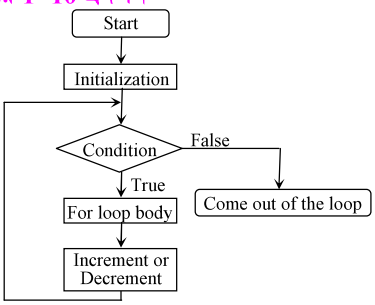
ধারাবাহিকভাবে কয়েকটি সংখ্যা প্রদর্শন করার প্রোগ্রাম :

প্রোগ্রাম-৩০ : 1–10 পর্যন্ত প্রদর্শন

```
#include<stdio.h>
int main()
{
int i;
for (i=1;i<11;i++)
 {
 printf("\n %d",i);
 }
return 0;
}
```

প্রোগ্রাম-৩১: goto ব্যবহার করে 1–10 প্রদর্শন

```
#include<stdio.h>
int main(){
int i=1;
start:
printf("%d ",i);
i=i+1;
if(i<=10)
goto start;
}
```



১ম প্রোগ্রামটি রান করালে ১ ২ ৩ ৪ ৫ ৬ ৭ ৮ ৯ ১০ প্রদর্শিত হবে। ২য় প্রোগ্রামটি রান করালে একই মান প্রদর্শিত হবে।

প্রোগ্রামে \n ইস্কেপ সিকুয়েন্স ব্যবহার করায় সংখ্যাগুলো পাশাপাশি প্রদর্শিত না হয়ে নতুন নতুন লাইনে প্রদর্শিত হবে।

প্রোগ্রাম-৩২ : মি. করিম ১/১/২০২৫ তারিখে একটি কোম্পানিতে চুক্তিভিত্তিক যোগদান করেন। তার চাকরী চুক্তিভিত্তিক হওয়ায় তাকে প্রতি ৭ দিনে ১ দিন অফিস করতে হবে। সি প্রোগ্রাম লিখ যা জানুয়ারি মাসের জন্য মি. করিমের এর অফিসের তারিখগুলো দেখাবে।

```
#include <stdio.h>
int main() {
   int day;
   printf("Office date of January 2025:\n");
   for(day = 1; day <= 31; day += 7) {
      printf("date: %d January 2025\n", day);
   }
   return 0;
}
```

Output:

```
Office date of January 2025:
date: 1 January 2025
date: 8 January 2025
date: 15 January 2025
date: 22 January 2025
date: 29 January 2025
```

প্রোগ্রাম-৩৩ : 10+9+8+7+ .... + Y ধারাটির যোগফল নির্ণয় কর।

```
#include <stdio.h>
int main() {
   int Y, sum = 0;
   printf(" Enter last number Y (which is less than or equal to 10): ");
   scanf("%d", &Y);
   if (Y > 10) {
      printf("Please provide a number that is equal to or less than 10.\n");
      return 1;
   }
   for (int i = 10; i >= Y; i--)
   {
      sum += i;
   }
   printf("sum = %d\n", sum);
   return 0;
}
```

Output:

```
Enter last number Y (which is less than or equal to 10): 4 ↵
sum = 49
```

for লুপ ব্যবহারের মাধ্যমে বিভিন্ন সমস্যা সমাধান

10 থেকে 1 পর্যন্ত উল্টোভাবে প্রিন্ট করা

```
for(int i = 10; i >= 1; i--) {
   printf("%d ", i);
}
```

1 থেকে 10 পর্যন্ত সংখ্যাগুলোর ঘনফল যোগ

```
int sum = 0;
for(int i = 1; i <= 10; i++) {
   sum += i*i*i;
}
printf("Sum of Cubes = %d", sum);
}
```

1 থেকে 10 পর্যন্ত জোড় সংখ্যা প্রিন্ট করা

```
for(int i = 2; i <= 10; i += 2) {
   printf("%d ", i);
}
```

১০ বার ইউজারের বয়স প্রিন্ট (same value)

```
int age = 16;
for(int i = 1; i <= 10; i++) {
   printf("Age: %d\n", age);
}
```

1 থেকে 9 পর্যন্ত বিজোড় সংখ্যা প্রিন্ট করা

```
for(int i = 1; i < 10; i += 2) {
   printf("%d ", i);
}
```

সংখ্যার সব ভাজক বের করা (ধরা যাক num = 24)

```
int num = 24;
for(int i = 1; i <= num; i++) {
   if(num % i == 0) printf("%d ", i);
}
```

1 থেকে 5 পর্যন্ত সংখ্যার বর্গ প্রিন্ট করা

```
for(int i = 1; i <= 5; i++) {
   printf("%d^2 = %d\n", i, i*i);
}
```

1 থেকে 100 পর্যন্ত শুধু 7 দ্বারা বিভাজ্য সংখ্যা প্রিন্ট করা

```
for(int i = 1; i <= 100; i++) {
   if(i % 7 == 0) printf("%d ", i);
}
```

1 থেকে 5 পর্যন্ত সংখ্যার গড় বের করা

```
for(int i = 1; i <= 5; i++) {
   sum += i;
}
float avg = sum / 5.0;
printf("Average = %.2f", avg);
```

1 থেকে 10 পর্যন্ত সংখ্যার যোগফল ও গড় একসাথে বের করা

```
int sum = 0;
for(int i = 1; i <= 10; i++)
{
   sum += i;
}
float avg = sum / 10.0;
printf("Sum = %d, Average = %.2f", sum,
avg);
```

ছোট হাতের অক্ষর a-z প্রিন্ট করা

```
for(char c = 'a'; c <= 'z'; c++)
{
   printf("%c ", c);
}
```

A থেকে E পর্যন্ত অক্ষর প্রিন্ট করা

```
for(char c = 'A'; c <= 'E'; c++)
{
   printf("%c ", c);
}
```

do...while লুপ

সি তে শর্ত সাপেক্ষে এক বা একাধিক বার কোন স্টেটমেন্ট সম্পাদনের জন্য do...while স্টেটমেন্ট ব্যবহৃত হয়। do স্টেটমেন্ট while স্টেটমেন্টের সাহায্য ছাড়া কাজ করে না। do...while লুপ স্টেটমেন্টকে অনেক সময় do স্টেটমেন্টও বলা হয়। while স্টেটমেন্টের মতো পূর্বে ঘোষিত কোন কাউন্টার ভেরিয়েবল ব্যবহার করে do স্টেটমেন্টের আবর্তন সংখ্যা গণনা করা হয় এবং পুনরাবর্তন সম্পর্কে সিদ্ধান্ত নেয়া হয়।

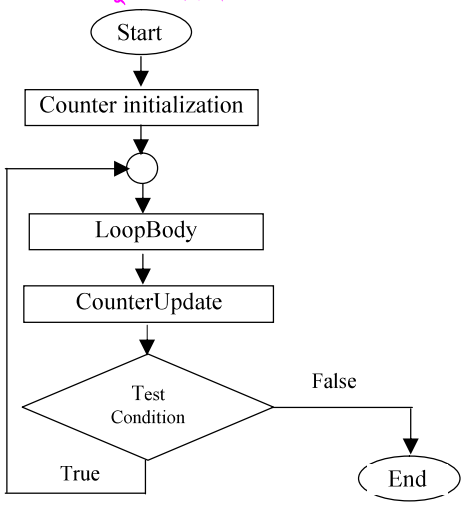
do...while লুপ নিয়ন্ত্রণের ফরম্যাট :

```
Counter Declaration;
Counter Initialization;
do
{
Statements;
Increment/Decrement;
} while(Condition);
```

```
int i=1;
do{
printf("Programming in C\n");
i=i+1;
}while (i<=5);
```

উদাহরণ: ধরা যাক, Programming in C লেখাটি স্ক্রিনে 10 বার প্রদর্শন করতে হবে। এক্ষেত্রে প্রোগ্রামে printf("Programming in C\n"); লেখাটি দশবার টাইপ না করে do...while লুপ ব্যবহার করে সহজেই করা যায়।

do- while লুপের প্রবাহচিত্র :



প্রোগ্রাম-৩৪ : a থেকে z পর্যন্ত পর্দায় প্রদর্শন করা।

```
# include<stdio.h>
int main()
{
char alphabet = 'a';
do{
printf("% c\t", alphabet ++);
}while (alphabet <= 'z');
return 0;
}
```

প্রোগ্রাম-৩৫ : 1 থেকে 20 পর্যন্ত সংখ্যাগুলো প্রদর্শন

```
# include<stdio.h>
int main ( ) {
int i = 1;
do
{
printf ("% d", i);
++ i;
}while (i < 21);
return 0;
}
```

প্রোগ্রাম-৩৬ : A-Z পর্যন্ত বড় হাতের অক্ষর প্রদর্শন

```
#include<stdio.h>
 int main(){
char ch;
do{
 printf("%c ", ch);
 ch++;
 }while((ch >= 'A') && (ch <= 'Z'));
return 0;
}
```

গারবেজ (Garbage) মান : একটি ফাংশনে যখন কোন ভেরিয়েবল ঘোষণা করা হয় (যেমন int i), তখন সেই ভেরিয়েবলের ভেতরে কোনো মান দেয়া থাকে না। ভেরিয়েবলটির ভেতরে যে কোন মান থাকতে পারে। একে গারবেজ মান বলা হয়।

While লুপ

While লুপে প্রথমেই শর্তের মান পরীক্ষিত হয়, শর্ত সত্য হলে তবেই লুপ বডি সম্পাদিত হয়। শর্ত সত্য না হলে while লুপের সাথে সংশ্লিষ্ট স্টেটমেন্ট সম্পাদিত হয় না। while লুপ-এর ফরম্যাট হলো-

```
while(condition)
  statement;
```

প্রোগ্রাম-৩৭ : While লুপ ব্যবহার করে 1 থেকে n পর্যন্ত বেজোড় সংখ্যা নির্ণয়ের একটি প্রোগ্রাম

```
# include<stdio.h>
int main ()
{
int digit=1,n;
printf("Enter n: ");
scanf("%d",&n);
printf("The odd numbers are :");
while (digit <= n)
{
if (digit %2 != 0)
{
printf("%d  ",digit);
}
++digit;
}
return 0;
}
```

Output:
Enter number (n) : 10 ↵
The Odd numbers are : 1 3 5 7 9

উক্ত প্রোগ্রামটিতে while লুপের সাথে দেয়া শর্তের মান পরীক্ষা করে দেখবে ইনপুট করা সংখ্যাটি digit ভেরিয়েবলের মানের চেয়ে ছোট অথবা সমান কিনা? শর্ত সত্য হলে তবেই লুপ বডি সম্পাদিত হবে অর্থাৎ digit ভেরিয়েবলের মানটিকে ২ দিয়ে ভাগ করে ভাগফল 0 না হলে বেজোড় সংখ্যাটি প্রিন্ট করবে। 0 হলে জোড় সংখ্যাটি প্রিন্ট করবে না।

প্রিন্ট করার পর digit ভেরিয়েবলের মান এক করে বাড়বে।

এখন আবার দেখবে ইনপুট করা সংখ্যাটি (n এর মান) digit ভেরিয়েবলের মানের চেয়ে ছোট অথবা সমান কিনা?

শর্ত সত্য হলে digit ভেরিয়েবলের মানটিকে 2 দিয়ে ভাগ করে ভাগফল 0 না হলে বেজোড় সংখ্যাটি প্রিন্ট করবে।

প্রিন্ট করার পর digit ভেরিয়েবলের মান 1 বাড়বে।

এভাবে চলতে থাকবে, যতক্ষণ while লুপের সাথে দেয়া শর্তটি সত্য হয়।

প্রোগ্রাম-৩৮ : 1+3+9+27+…+N ধারাটির যোগফল নির্ণয় কর।

```
#include <stdio.h>
int main() {
   int term = 1;        // □□□□□□ □□□□□ পদ 1
   int sum = 0;         // □□□ □□□□□
   int N;
   printf("□□□ পদ (N) □□□□□ □□□: "); // □□□□□ □□□□□
   scanf("%d", &N);
   while (term <= N) {    // □□□□□□ term <= N, □□□□□□ □□□ □□□□
      sum += term;        // □□□□□□ □□□□□□□ term □□□
      term = term * 3;    // □□□□□□□ term □□□ □□□□ ৩ □□□
   }
   printf("১ +৩+৯+ ... + %d □□□□□□□ □□□□□ = %d\n", N, sum);  // □□□□□
□□□□□□□
   return 0;
}
```

এই ধারা একটি জ্যামিতিক ধারা (Geometric Series) যার প্রথম পদ: a = 1, গুণোত্তর অনুপাত: r = 3 শেষ পদ: N (যেমন: 1, 3, 9, 27, ..., N)
আমরা জানি, যদি N পর্যন্ত পদগুলো 3-এর ঘাত আকারে থাকে (1, 3, 9, 27, ..., N), তাহলে এই ধারা এমন: S=1+3+9+27+⋯+N

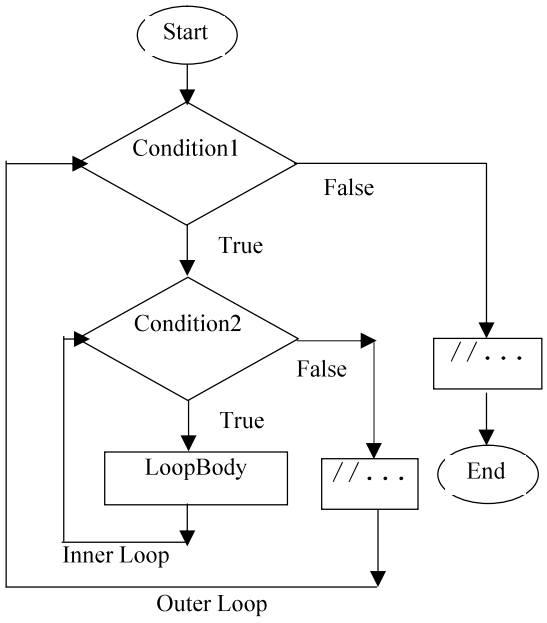
While ও Do while লুপ-এর মধ্যে পার্থক্য

While লুপ	Do while লুপ
১. While লুপ-এর শুরুতে দেয়া শর্ত পরীক্ষা করা হয়।	১. Do while লুপ-এর শেষে দেয়া শর্ত পরীক্ষা করা হয়।
২. শুরুতে দেয়া শর্ত সঠিক হলে While লুপ–এর ভেতরে থাকা স্টেটমেন্টসমূহ নির্বাহ হয়।	২. যেহেতু Do while লুপ–এর শর্ত শেষে থাকে, তাই Do while লুপ–এর ভেতরে থাকা স্টেটমেন্টসমূহ কমপক্ষে একবার নির্বাহ হয়।
৩. শর্ত সঠিক না হলে While লুপ–এর ভেতরে থাকা স্টেটমেন্টসমূহ নির্বাহ হয় না।	৩. শর্ত সঠিক না হলেও Do while লুপ–এর ভেতরে থাকা স্টেটমেন্টসমূহ কমপক্ষে একবার নির্বাহ হবে।
৪. While লুপ–এর শেষে কোনো সেমিকোলন থাকে না।	৪. Do while লুপে While লুপের শর্তের শেষে সেমিকোলন থাকে।

৫. While লুপ-এর সিনটেক্স :
while (condition)
{
Statements;
}
	৫. Do while লুপ-এর সিনটেক্স :
	do
	{
	Statements;
	}while (condition);

নেস্টেড for লুপ

যখন একটি for লুপের মধ্যে অপর একটি for লুপ থাকে, তখন মধ্যবর্তী for লুপকে নেস্টেড for লুপ বলা হয়। এক্ষেত্রে মধ্যবর্তী for লুপকে ইনার for লুপ, বহিস্থ for লুপকে আউটার for লুপ বলা হয়। আউটার for লুপের পূর্বে ইনার for লুপের কাজ শেষ হয়। চিত্রের মাধ্যমে নেস্টেড লুপ স্টেটমেন্টের প্রবাহচিত্র দেখানো হলো :




```
/* Displaying a triangular  series */
#include<stdio.h>
int main()
{
int i=0, j=0;
for (i=1; i<=5; i++)          // outer for loop
{
for (j=1; j<=i; j++)          // inner for loop
{
printf("%d  ", j);
}                             // End of inner for loop
printf("\n");
}                             // End of outer for loop
}
```

Output:

```
1
1  2
1  2  3
1  2  3  4
1  2  3  4  5
```

প্রোগ্রাম-৩৯: কতগুলো মৌলিক সংখ্যা নির্ণয় করে ফলাফল প্রদর্শন

```
#include<stdio.h>
#include<conio.h>
void main()
{
int i, N, d;
int isprime;
printf("\nEnter the highest range : ");
scanf("%d", &N);
printf("Series of prime number upto %d is:\n", N);
for(i=2; i<=N; i++)
{
isprime = 1;
for(d=2; d<i; d++)      // Inner for
if(i%d ==0)
isprime =0;
if(isprime !=0)
printf("%d  ", i);
}                       // End of for
getch();
}                       // End of main()
```

যে সব সংখ্যা 1 এবং সেই সংখ্যা ভিন্ন অন্য কোনো সংখ্যা দিয়ে নিঃশেষে বিভাজ্য নয়, সেগুলোকে মৌলিক সংখ্যা বলা হয়। প্রথমে scanf() ব্যবহার করে সিরিজের সর্বশেষ মান গ্রহণ করতে হবে। অতঃপর যেহেতু মৌলিক সংখ্যাসমূহ 2 থেকে শুরু হয়, তাই 2 হতে শুরু করে সংখ্যাটির অর্ধেক পর্যন্ত পর্যায়ক্রমিক পূর্ণসংখ্যা দিয়ে সংখ্যাটিকে ভাগ করতে হবে, কোনো ক্ষেত্রেই ভাগশেষ শূন্য না হলে সংখ্যাটি মৌলিক হিসেবে বিবেচিত হবে।

এভাবে প্রোগ্রামে লুপ স্টেটমেন্ট ব্যবহার করে পর্যায়ক্রমিক মৌলিক সংখ্যা নির্ণয় করা যাবে।

Output:

```
Enter the highest range : 25 ↵
Series of prime number upto 25 is:
 2  3  5 7  11 13  17 19  23
```

ব্রাঞ্চিং স্টেটমেন্ট goto এর ব্যবহার

সি/সি++ প্রোগ্রামে শর্তযুক্ত অথবা শর্তবিহীনভাবে এক স্টেটমেন্ট থেকে উপরে বা নিচে অপর কোনো স্টেটমেন্টে স্থানান্তরের জন্য goto স্টেটমেন্ট ব্যবহৃত হয়। সি প্রোগ্রামে goto স্টেটমেন্ট জাম্প (jump) স্টেটমেন্ট হিসেবে পরিচিত। goto স্টেটমেন্টের ফরম্যাট হলো :

```
goto  LevelName;
  ---  ---  ----
  ---  ---  ----
LevelName :
  ---  ---  ----
  ---  ---  ----
[ ফরওয়ার্ড জাম্পিং ]
```

দুই ধরনের goto স্টেটমেন্ট রয়েছে। যথা—

১. Forward Jumping
২. Backward Jumping

১. Forward Jumping : goto স্টেটমেন্টের পরে লেভেল ডিক্লেয়ার করলে ফরওয়ার্ড জাম্পিং হয়।

২. Backward Jumping: goto স্টেটমেন্টের আগে লেভেল ডিক্লেয়ার করলে ব্যাকওয়ার্ড জাম্পিং হয়।

```
LevelName :
  ---  ---  ----
  ---  ---  ----
goto LevelName;
  ---  ---  ----
  ---  ---  ----
[ ব্যাকওয়ার্ড জাম্পিং ]
```

এখানে, LevelName প্রোগ্রামার কর্তৃক দেয়া প্রোগ্রামের কোনো স্থান নির্দেশক নাম। LevelName একটি আইডেন্টিফায়ার। LevelName এর শেষে একটি কোলন দিতে হয়। একই প্রোগ্রাম বা ফাংশনে প্রয়োজনে ভিন্ন ভিন্ন নামে একাধিক লেভেল স্টেটমেন্ট ব্যবহার করা যেতে পারে। লেভেল স্টেটমেন্ট goto স্টেটমেন্টের উপরে বা নিচে ব্যবহার করা যেতে পারে। goto ব্যবহার করে প্রোগ্রামের সামনে কিংবা পিছনে যে কোনো স্থানে স্থানান্তর বা জাম্প সম্ভব। goto স্টেটমেন্ট ব্যবহার করে if, else if স্টেটমেন্টের সাথে সম্পর্কিত শর্ত সাপেক্ষে প্রোগ্রামের অপর কোনো স্থানে জাম্প করা হয়। নিচের উদাহরণে goto স্টেটমেন্টের ব্যবহার দেখানো হয়েছে।

প্রোগ্রাম-৪০ (উদাহরণ-১) :

```
#include<stdio.h>
int main() {
 printf ("Hello World\n");
 goto A;
 printf ("How are you?");
 printf ("Are you Okey?");
 A :
 printf ("Hope you are fine");
 return 0;
}
```

আউটপুট :

Hello World
Hope you are fine

এখানে Hello World প্রিন্ট হওয়ার পর goto A; পাওয়াতে পরের দুইটি লাইন প্রিন্ট না হয়ে এ দুইটি লাইন স্কিপ করে A লেভেলে চলে যাবে এবং সেখানের Hope you are fine প্রিন্ট হবে। এভাবে goto এর সাথে যে লেভেল থাকবে, সে লেভেলে জাম্প করবে।

প্রোগ্রাম-৪১ (উদাহরণ-২) :

```
#include<stdio.h>
int main(){
int i=10;
start:
printf ("%d ",i );
i=i+10;
if (i<100)
goto start;
return 0;
}
```

আউটপুট :

10 20 30 40 50 60 70 80 90 100

ব্যাখ্যা :

- যেহেতু 10-100 এর মধ্যেই সংখ্যা দেখানো হবে, তাই ১ম সংখ্যা i=10 ধরা হয়েছে।
- 10 এর পর 20, তারপর 30 এভাবে 10 অন্তর বুঝানোর জন্য i=i+10 লেখা হয়েছে।
- i এর মান 100 এর কম বা সমান হলে goto start এর কারণে start : এ যাবে এবং শর্তানুযায়ী শেষ সংখ্যার সাথে ১০ যোগ হয়ে সংখ্যাটি প্রদর্শন করে আবার শর্ত পরীক্ষা করবে।

প্রোগ্রাম-৪২ : সংখ্যার বর্গমূল নির্ণয়

```
//Example of continue control Statement*/
/* Example of goto statement*/

#include<stdio.h>
#include<math.h>
int main(){
int n1,n2;
Label:
printf("Enter a number:");
scanf("%d",&n1);
 if(n1>0)
 n2 = sqrt(n1);
printf("Square root of %d is %d\n",n1,n2);
goto Label;
return 0;
}
```

আউটপুট :

```
Enter a number: 81 ↵

Square root of 81 is 9 ↵
```

প্রোগ্রাম-৪৩ : সংখ্যার ফ্যাক্টরিয়াল নির্ণয়

```
#include<stdio.h>
int main(){
int x; long fact=1;
HASAN:
printf("\nEnter a positive integer:");
scanf("%d",&x);
if(x<0) {
print("nNegative Number not allowed");
goto HASAN;
}
else if (x= =0)
printf("\nFactorial of %d is: %1d",x,fact);
else
{
for (i=1; i<=x; i++)
fact=fact*i;
printf("\nFactorial of %d is: %1d", x, fact);
}
}
```

আউটপুট :

```
Enter a positive integer: -5 ↵
Negative Number not allowed.
Enter a positive integer: 5 ↵
Factorial of 5 is: 120
```

প্রোগ্রাম-৪৪ : goto স্টেটমেন্ট ব্যবহার করে 1+2+3.. ... +N ধারার যোগফল নির্ণয়।

```
#include<stdio.h>
int main()
{
int n,i,sum=0;
printf("Enter last value :");
scanf("%d",&n);
i=1;
level: sum=sum+i;
i=i+1;
if(i<=n) goto level;
printf("sum of series :%d",sum);
return 0;
}
```

Output :
Enter last value : 100 ↵
Sum of series : 5050

প্রোগ্রাম-৪৫ : goto স্টেটমেন্ট ব্যবহার করে 1+3+5.. ... +N ধারার যোগফল নির্ণয়।

```
#include<stdio.h>
int main()
{
int n,i,sum=0;
printf("Enter last value :");
scanf("%d",&n);
i=1;
level: sum=sum+i;
i=i+2;
if(i<=n) goto level;
printf("sum of series :%d",sum);
return 0;
}
```

Output :
Enter the value of n: 10 ↵
Sum of series : 25